

尚硅谷嵌入式之嵌入式 Linux 移植

(作者：尚硅谷研究院)

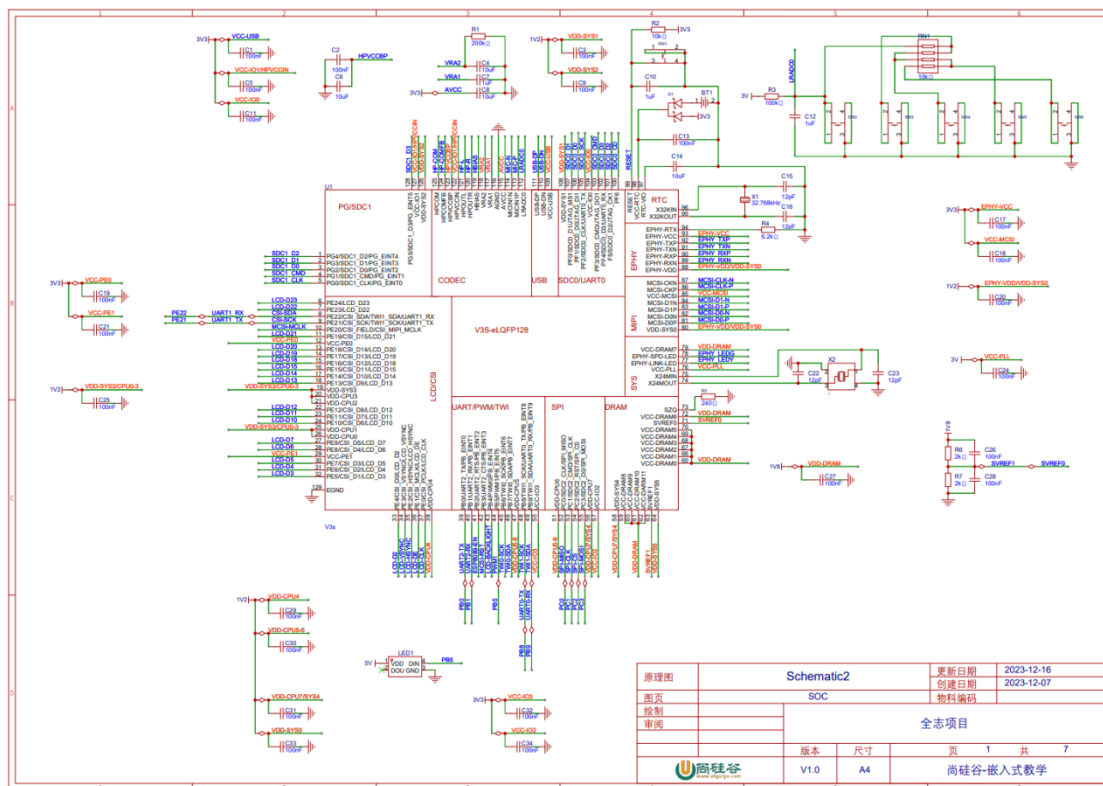
版本：V1.0

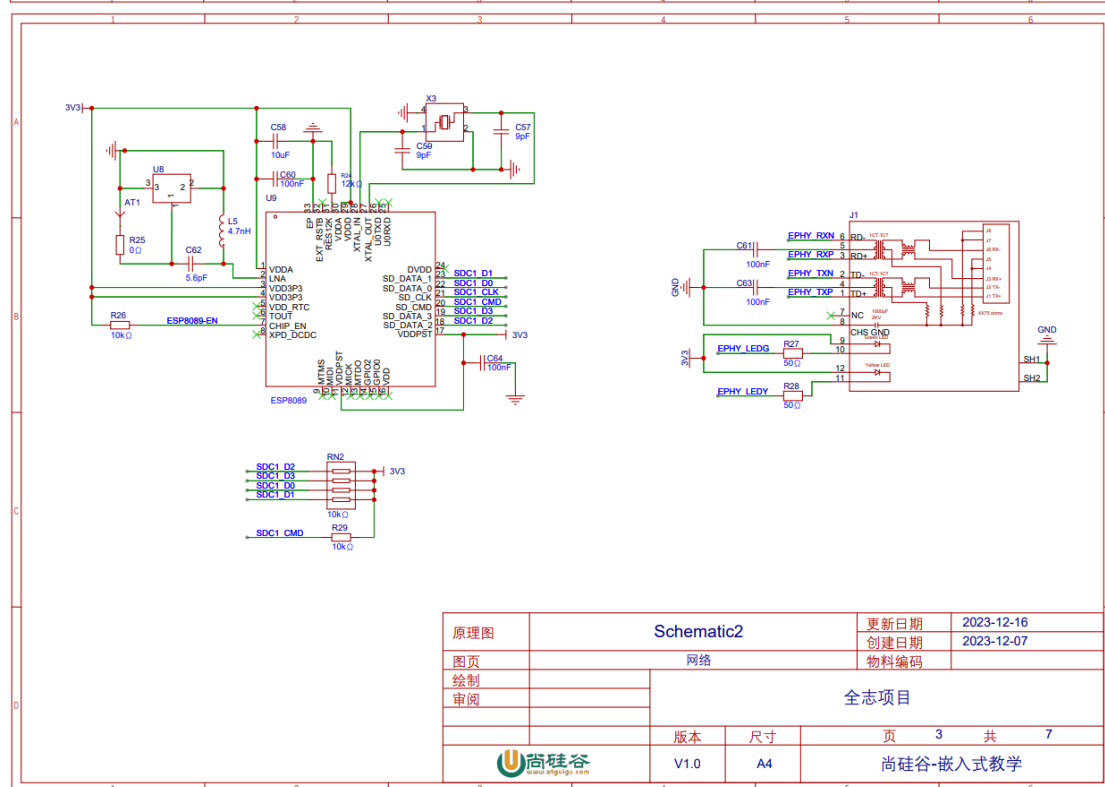
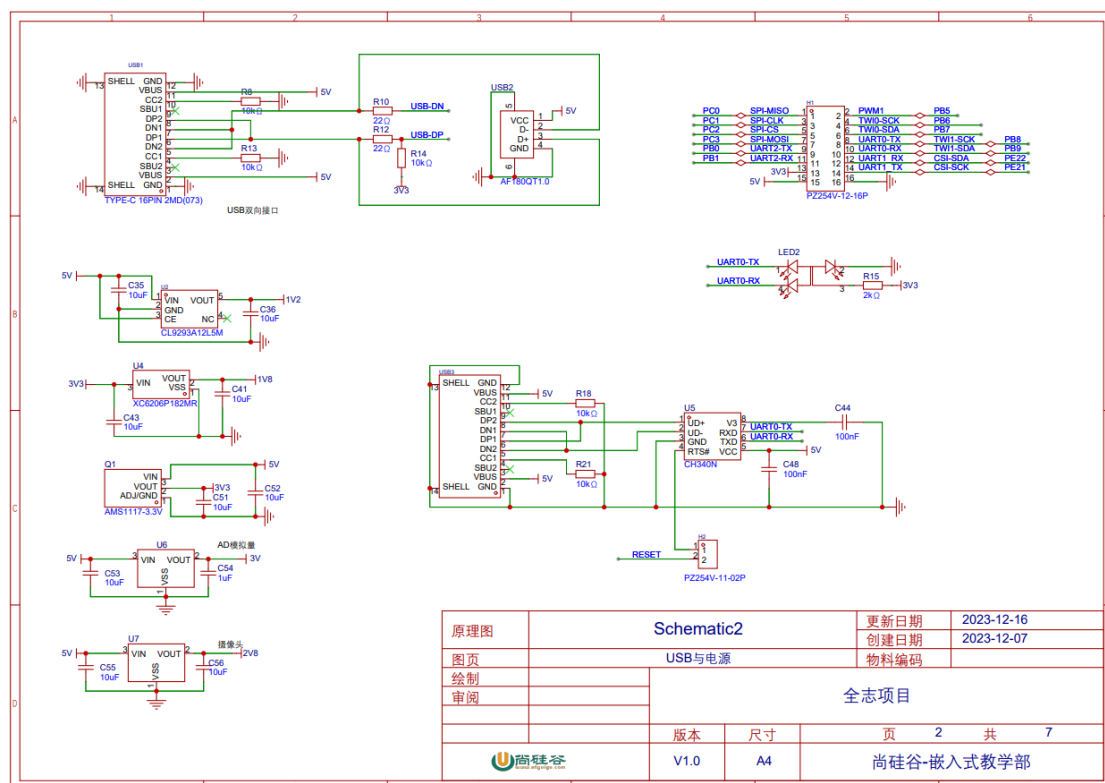
第 1 章 嵌入式 Linux 简介

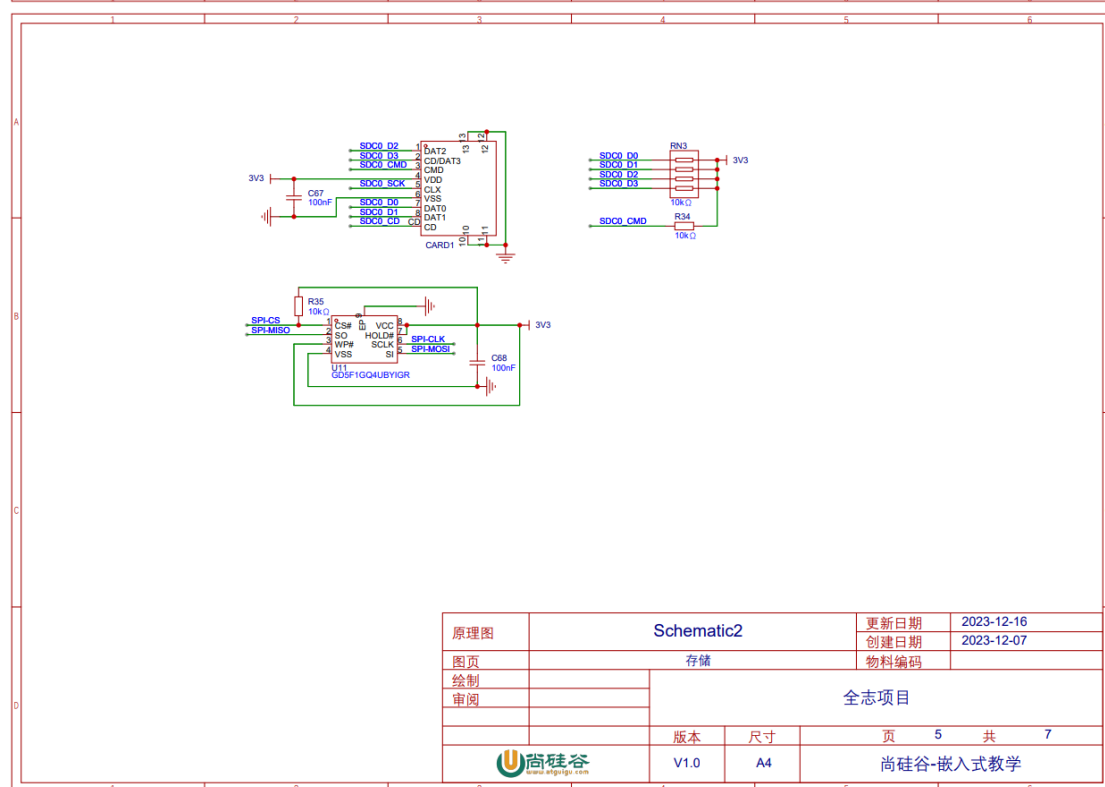
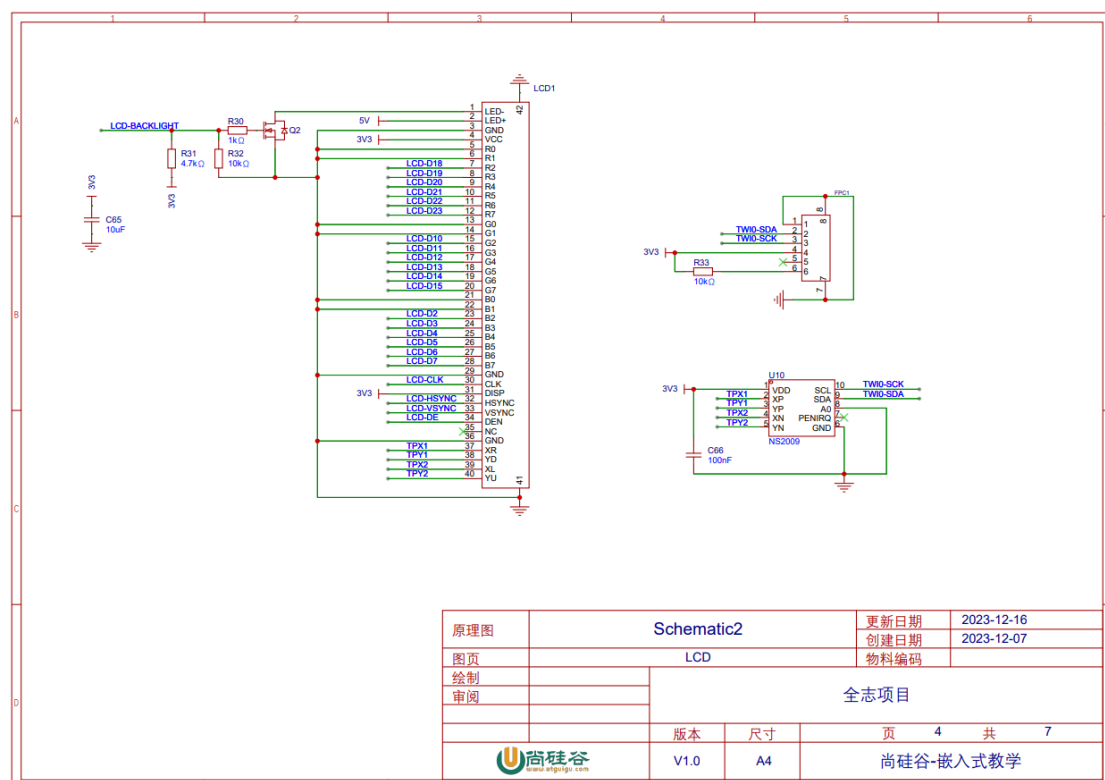
嵌入式 Linux 是将 Linux 操作系统移植到嵌入式系统中的一种应用形式，通常用于资源有限的嵌入式设备，如智能家居、工业自动化和车载系统等。其技术特点包括开源、灵活、可定制性高、稳定性强以及丰富的开发工具和生态系统支持。嵌入式 Linux 可应用于各种场景，如嵌入式设备控制、实时数据采集、嵌入式图像处理等，为嵌入式系统提供了强大的操作系统支持和开发平台。

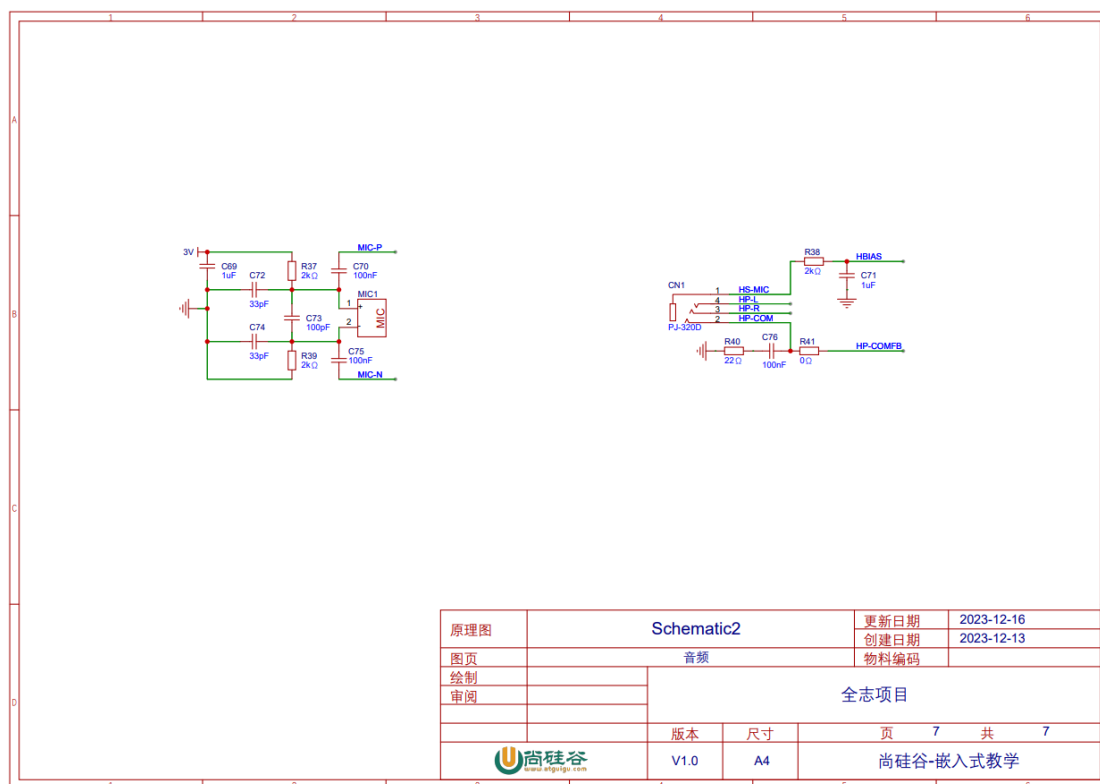
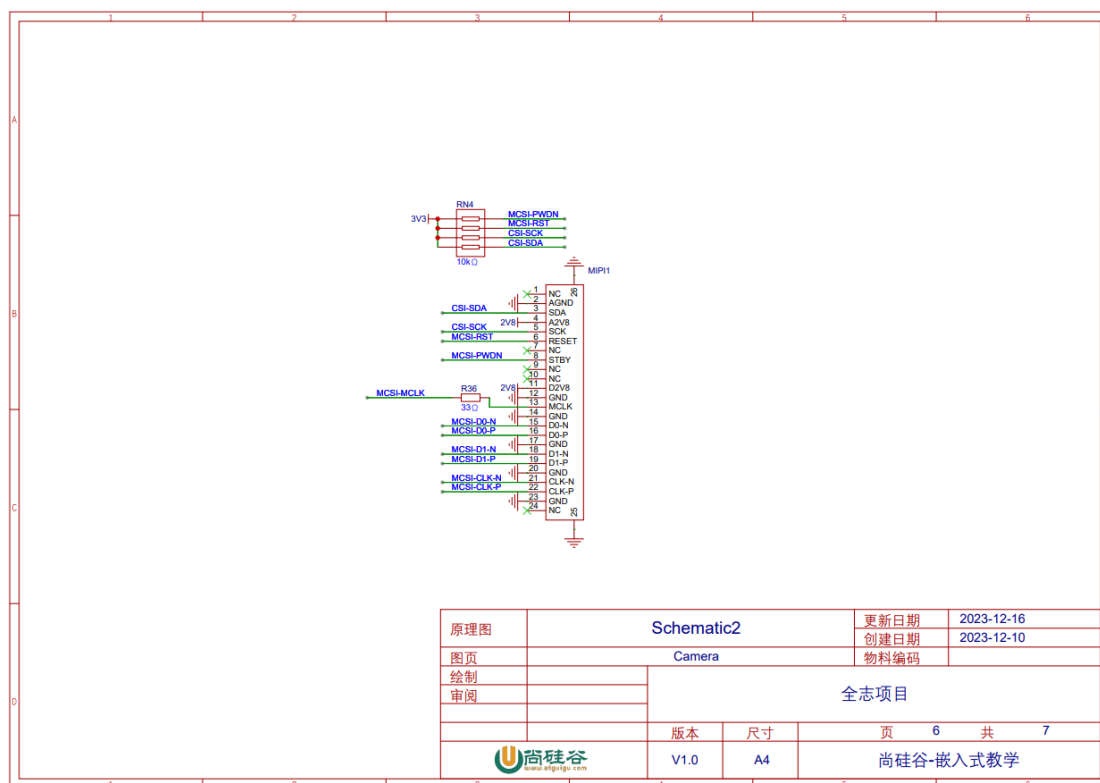
第 2 章 硬件平台

我们的嵌入式 Linux 开发板是一个以全志 V3s 为核心的平台，硬件原理图如下：





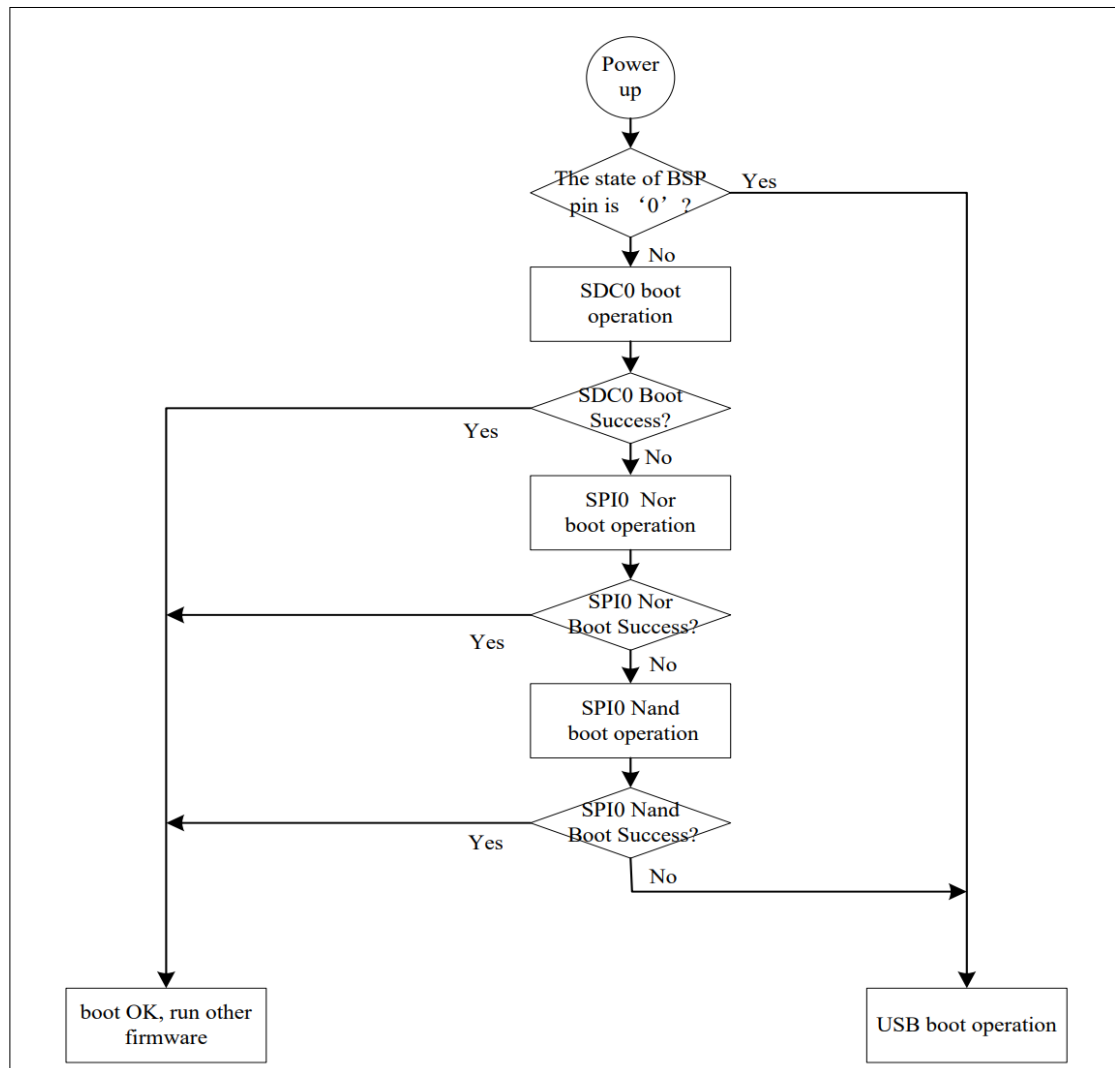




第 3 章 准备工作

3.1 全志 v3s 引导流程

全志 V3s 在上电之后首先会执行芯片内置的 BROM 区。根据芯片手册的说明，这一段代码总共 32K，内置了一段引导程序，逻辑如下图所示：



芯片上电后首先会检查 BSP 引脚电平，如果为 0 则直接 USB 引导；

否则挨个尝试从 SD 卡，Nor flash，Nand Flash 引导，都不成功最后也会回落到 USB 引导。

1) SD 卡引导

引导程序从 SD 卡引导时，会从 SD 卡 8k offset 开始继续执行，所以我们需要将二阶段引导程序（u-boot）写入到 SD 卡的 8k 位置。由于我们不采用 SD 卡引导，这里不做详细介绍，有兴趣的同学可以关注我们后续放出的资料。

2) Flash 引导

这是我们之后要采取的引导方案。BROM 引导到 Flash 后，会去 offset 0 位置读取 u-boot。在找到 u-boot 后，就进入了通用的引导流程：u-boot 会将 Linux 的设备树和内核加载

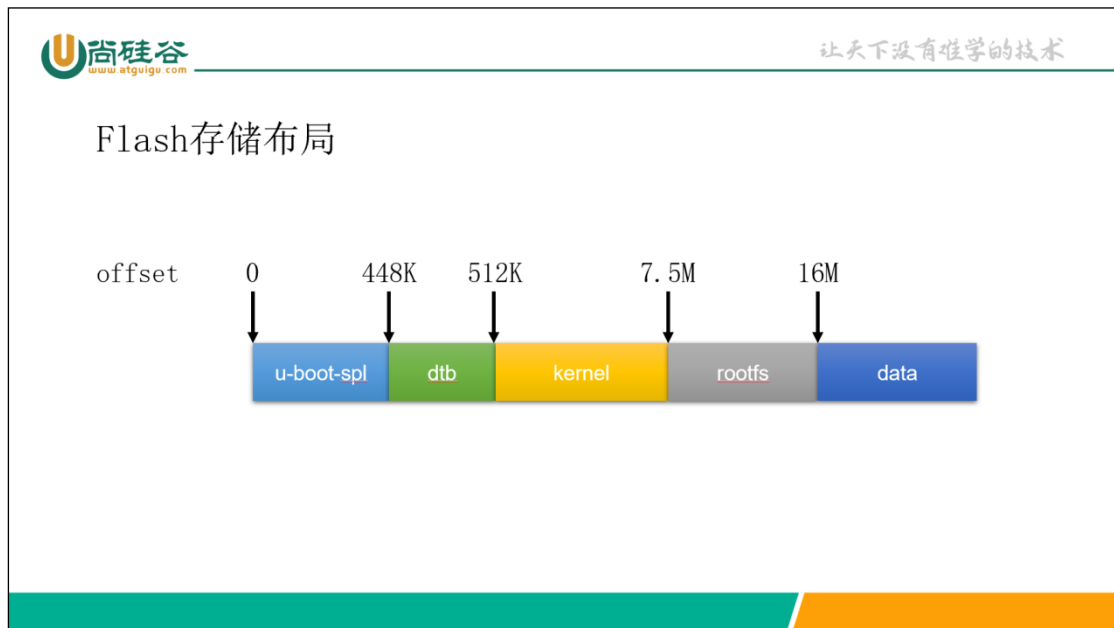
到内存并执行，内核最后会挂载 rootfs 和 data 分区。这其中关键的部分就是做好 Flash 存储布局，引导的配置和布局要能对的上。Flash 的详细布局在下一节。

3) USB 引导

当所有启动方式不奏效时，V3s 就会进入 USB 引导流程，此时我们可以用 fel 工具和芯片交互，可以直接从 usb 读取 uboot 引导，可以向 Flash 芯片刷机。

3.2 Flash 存储布局

我们的 Flash 总共 32M 大小，按照以下布局划分：

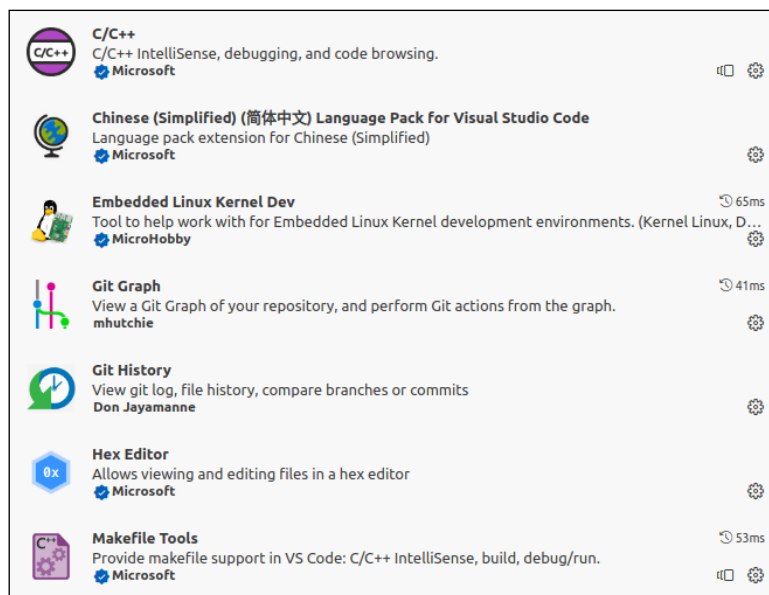


3.3 在 Ubuntu 中安装并配置 VSCode

与 Windows 类似，我们之后的工作需要使用 VSCode 进行开发，所以我们首先需要在 Ubuntu 中安装 VSCode。打开终端执行以下指令：

```
atguigu@ubuntu:~$ sudo apt install -y code
```

安装完成后配置 VSCode，我们需要安装以下插件：



插件安装完成后，VSCode 环境就准备完毕了

3.4 交叉编译环境准备

我们需要在 x86 Linux 中编译可以用于全志 v3s 的 u-boot、Linux 内核（Kernel）与根文件系统（rootfs），这就需要用到一种技术：交叉编译。

交叉编译，就是将源代码编译成不是本地指令集执行的二进制文件。我们目前是需要是在 x86 环境下编译 arm 指令集的二进制，所需的工具链为 gcc-arm-linux-gnueabi。此外，为了能够顺利完成编译工作，我们还需要一些其他的工具。

以下的命令基于 Ubuntu 22.04 LTS，打开终端执行下列命令：

1) 安装必要环境

```
atguigu@ubuntu:~$ sudo apt install git bison flex libncurses-dev build-essential python3-setuptools libssl-dev swig mtd-utils python3-dev universal-ctags libfdt-dev
```

2) 创建工作空间

```
atguigu@ubuntu:~$ mkdir atguigupi
```

3) 创建最终结果空间

```
atguigu@ubuntu:~$ mkdir atguigupi/output
```

3.5 准备工具链

首先将资料文件夹中的 gcc-linaro-12.3.1-2023.06-x86_64_arm-linux-gnueabi.tar.xz 解压到 atguigupi 目录，并将其重命名为 toolchain。这就是我们将来用到编译 u-boot，内核，rootfs 的交叉编译工具链。

```
atguigu@ubuntu:~$ cd atguigupi
```

```
atguigu@ubuntu:~/atguigupi$ tar -xf gcc-linaro-12.3.1-2023.06-x86_64_arm-linux-gnueabihf.tar.xz
atguigu@ubuntu:~/atguigupi$ mv gcc-linaro-12.3.1-2023.06-x86_64_arm-linux-gnueabihf toolchain
```

该工具链是由 linaro 发布，linaro 是一个非盈利性值的开源组织，主要目标是开发不同半导体芯片之间的通用工具。其主页为：<https://www.linaro.org>

第 4 章 U-boot 移植

4.1 下载 U-boot 源码

U-boot 源代码地址：<https://github.com/u-boot/u-boot>

我们选择 master 分支的 2024.01 tag 作为源码。获取源码的方式有两种：

1) 使用 git 命令拉取源码

首先拉取源码：

```
atguigu@ubuntu:~/atguigupi$ git clone https://github.com/u-boot/u-boot.git
```

再切换 tag：

```
atguigu@ubuntu:~/atguigupi$ cd u-boot
atguigu@ubuntu:~/atguigupi/u-boot$ git checkout v2024.01
```

2) 直接下载源码压缩包

上一种方式会将仓库所有代码下载，体积会比较大，实际上我们只需要 v2024.01 这个分支的代码，所以我们可以直接下载该 tag 的源码并解压。

```
atguigu@ubuntu:~/atguigupi$ wget https://github.com/u-boot/u-boot/archive/refs/tags/v2024.01.tar.gz
atguigu@ubuntu:~/atguigupi$ tar -xf v2024.01.tar.gz
atguigu@ubuntu:~/atguigupi$ mv u-boot-2024.01 u-boot
```

之后我们对下载的代码初始化一个新的 git 仓库，方便后续修改源码

```
atguigu@ubuntu:~/atguigupi$ cd u-boot
atguigu@ubuntu:~/atguigupi/u-boot$ git init
atguigu@ubuntu:~/atguigupi/u-boot$ git add .
atguigu@ubuntu:~/atguigupi/u-boot$ git commit -m "v2024.01"
```

4.2 创建设备树文件

4.2.1 设备树文件简介

我们在单片机开发过程中，写过很多驱动。驱动代码实际上包含两个部分：硬件声明和驱动逻辑。硬件声明就是和硬件绑定的部分，例如我们写一个软件 IIC 驱动，需要提前声明 IIC 的引脚编号，引脚工作模式等。驱动逻辑就是驱动具体运行的代码，还是以软件 IIC 为例，我们在代码中把引脚按照一定时序拉高拉低，就属于驱动逻辑代码。

嵌入式 Linux 作为一个操作系统，其正常工作的前提是能够正常驱动硬件工作。驱动硬件工作就需要驱动程序，然而不同于我们为特定单片机开发的驱动程序，Linux 是一个通用操作系统，它需要正常工作在各种各样的硬件上，而这些硬件都具有不同的硬件结构，这就带来一个问题：在 A 硬件上能够正常工作的驱动，在 B 硬件上就不能正常工作了，我们需要针对 B 硬件重新进行驱动开发工作，这些驱动代码都会合并到 Linux 内核主分支。硬件种类繁多多样，造成 Linux 内核代码臃肿不堪。于是在 Linux 3.x 时代，引入了设备树文件。

设备树文件其实就是硬件的硬件声明部分。有了设备树文件，我们就可以将硬件声明和驱动逻辑部分解耦。同是 IIC 驱动，在 A 和 B 两款硬件上的差别，可能仅仅就是硬件声明不同，而驱动逻辑的层面是通用的。在 A 上能够正常工作的 IIC 驱动，我们只要在 B 上针对其写一个设备树文件重新映射硬件，即可完成驱动移植。

4.2.2 编写设备树文件

我们使用全志 V3s 作为 SOC，V3s 的内置外设我们都需要在设备树中声明，不过我们可以不用从零编写设备树，而是找一个同样是 V3s SOC 的硬件，基于它的设备树进行修改。一般来说芯片厂商在发布芯片的时候，都会同步发布一张验证板，也会同步发布板子的设备树和驱动程序源码，如果我们基于这款芯片进行开发，就可以基于这些验证板的设备树和驱动进行二次开发。

这里我们采用同样使用全志 V3s 核心的荔枝派 zero 的设备树和驱动作为基础进行二次开发，首先我们将它的设备树文件复制一份并改名：

```
atguigu@ubuntu:~/atguigupi/u-boot$ cd arch/arm/dts
atguigu@ubuntu:~/atguigupi/u-boot/arch/arm/dts$ cp sun8i-v3s-licheepi-zero.dts sun8i-v3s-atguigupi.dts
```

VSCode 打开 u-boot 项目，打开 sun8i-v3s-atguigupi.dts 文件，观察里面的内容

```
/**
 * 首先声明两个头文件：
 * （1）sun8i-v3s.dtsi 是 v3s 这款 SOC 的通用声明，
 * 所有使用了这款 SOC 的开发板设备树声明都需要引用这个头文件。
 * 这个头文件里面主要是定义了一些 SOC 的片上外设声明，但是都处于失效状态，
 * 我们如果使用了哪个外设，并不需要从 0 开始写设备树，直接应用这个头文件启用设备即可。
 * （2）sunxi-common-regulators.dtsi 这个文件主要是声明了一些全志芯片公用的电压调节器。
 */
/dts-v1/;
#include "sun8i-v3s.dtsi"
#include "sunxi-common-regulators.dtsi"
```

```
/**
 * 这段代码开始从根声明设备树，首先定义了开发板型号，
 * 并声明了该开发板的兼容性：首选适配"licheepi,licheepi-zero"的驱动，
 * 其次选择适配"allwinner,sun8i-v3s"的驱动。
 */
/ {
    model = "Lichee Pi Zero";
    compatible = "licheepi,licheepi-zero", "allwinner,sun8i-v3s";
// 声明一个别名: serial0 指代 uart0 设备
    aliases {
        serial0 = &uart0;
    };
// 向内核传递参数，标准输出输出到串口 0，115200 波特率，8bit
    chosen {
        stdout-path = "serial0:115200n8";
    };
// 声明一个彩色 LED，兼容驱动为 gpio-leds，由 PG1，PG0，PG2 三个引脚控制，低电平开启
    leds {
        compatible = "gpio-leds";

        blue_led {
            label = "licheepi:blue:usr";
            gpios = <&pio 6 1 GPIO_ACTIVE_LOW>; /* PG1 */
        };

        green_led {
            label = "licheepi:green:usr";
            gpios = <&pio 6 0 GPIO_ACTIVE_LOW>; /* PG0 */
            default-state = "on";
        };

        red_led {
            label = "licheepi:red:usr";
            gpios = <&pio 6 2 GPIO_ACTIVE_LOW>; /* PG2 */
        };
    };
};
// 开启芯片的 mmc0 接口，这个接口的详细参数在 sun8i-v3s.dtsi 已经声明
// 这里只做了基本属性补充
// 这个外设非常关键，我们后面 u-boot 从 tf 卡引导用的就是这个外设
&mmc0 {
    // 没有卡检测引脚
    broken-cd;
```

```
// 总线宽度 4
bus-width = <4>;
// 电压调节器使用 sunxi-common-regulators.dtsi 声明的 3.3v 调节器
vmmc-supply = <&reg_vcc3v3>;
// 开启外设
status = "okay";
};
// 开启 uart0 设备, 就是第一个串口
// 这个设备也很重要, 因为之后我们所有的标准输出都是流向 uart0 的
&uart0 {
    // 串口引脚采用 sun8i-v3s.dtsi 里面声明的引脚
    pinctrl-0 = <&uart0_pb_pins>;
    // 引脚名字默认
    pinctrl-names = "default";
    // 开启外设
    status = "okay";
};
// 开启 usb_otg 外设
&usb_otg {
    dr_mode = "otg";
    status = "okay";
};

// 开启 usb 物理控制器
&usbphy {
    usb0_id_det-gpios = <&pio 5 6 GPIO_ACTIVE_HIGH>;
    status = "okay";
};
```

这个设备树文件中, 已经声明了重要的两个部分: mmc0 和 uart0, 除此之外没有声明 spi-flash 部分。mmc0 指的是 TF 控制器, 我们的 Linux 之后如果想从 SD 卡启动, 需要声明这个设备。uart0 是调试接口, 同样需要声明。我们实际打算将 Linux 安装到 spi-flash 上, 还需要声明一个 spi-flash 控制器。其他的设备 (包括 LED 和 USB 设备) 在 u-boot 阶段并不是很关键, 其实声明或者不声明都没什么影响。最终设备树文件如下:

```
/dts-v1/;
#include "sun8i-v3s.dtsi"
#include "sunxi-common-regulators.dtsi"

/ {
    model = "Atguigu Pi";
    compatible = "atguigu,atguigupi", "allwinner,sun8i-v3s";

    aliases {
```

```
    serial0 = &uart0;
    // 开启 spi 设备别名
    spi0 = &spi0;

};

chosen {
    stdout-path = "serial0:115200n8";
};

};

&mmc0 {
    broken-cd;
    bus-width = <4>;
    vmmc-supply = <&reg_vcc3v3>;
    status = "okay";
};

&uart0 {
    pinctrl-0 = <&uart0_pb_pins>;
    pinctrl-names = "default";
    status = "okay";
};

// 开启 spi 设备
&spi0 {
    status = "okay";
    // 声明总线最高频率
    spi-max-frequency = <100000000>;
    // 在 spi 总线上声明 flash 芯片
    w25q256:w25q256@0 {
        compatible = "winbond,w25q256", "jedec,spi-nor";
        reg = <0x0>;
        // 声明芯片最高频率
        spi-max-frequency = <50000000>;
        #address-cells = <1>;
        #size-cells = <1>;
    };
};
```

4.3 配置 menuconfig

在 configs 文件夹中有很多开发板的编译配置文件，里面设置了很多预定义宏，通过板级特定设置可以精确裁剪并编译用于特定开发板的二进制文件。我们同样从荔枝派 Zero 的配置文件进行二次修改，首先我们加载荔枝派的配置文件并进入配置菜单：

```
skiinder@ubuntu:~/atguigupi/u-boot$ make LicheePi_Zero_defconfig
skiinder@ubuntu:~/atguigupi/u-boot$ make menuconfig
```

荔枝派的配置文件和我们不同的地方在于

- 设备树不同
- 没有开启 spi-flash 支持

1) 使用我们自己的设备树文件

```
Device Tree Control --->
(sun8i-v3s-atguigupi) Default Device Tree for DT control
```

2) 开启 SPI-Flash 驱动支持

开启 flash 支持要稍微麻烦一点，因为最新版的 u-boot 对于 v3s 芯片的配置支持有一点问题，我们首先需要修复这个问题。

用 vscode 打开 arch/arm/mach-sunxi/Kconfig 文件，按照下图修改：

1063	
1064	_H3_H5 MACH_SUN50I MACH_SUN8I_R40 SUN50I_GEN_H6 MACH_SUNIV MACH_SUN8I_V3S
1065	
1066	rom
1067	loes

在1064行末尾添加上面的字符串

```
1064 depends on MACH_SUN4I || MACH_SUN5I || MACH_SUN7I ||
MACH_SUNXI_H3_H5 || MACH_SUN50I || MACH_SUN8I_R40 || SUN50I_GEN_H6 ||
MACH_SUNIV || MACH_SUN8I_V3S
```

进行上述修改后，我们才能够在 menuconfig 菜单中打开 SPL 阶段的 SPI-Flash 支持。按照下面的配置修改：

```
// 开启 SPL 阶段能从 FLASH 启动 U-Boot
ARM architecture --->
[*] Support for SPI Flash on Allwinner SoCs in SPL
```

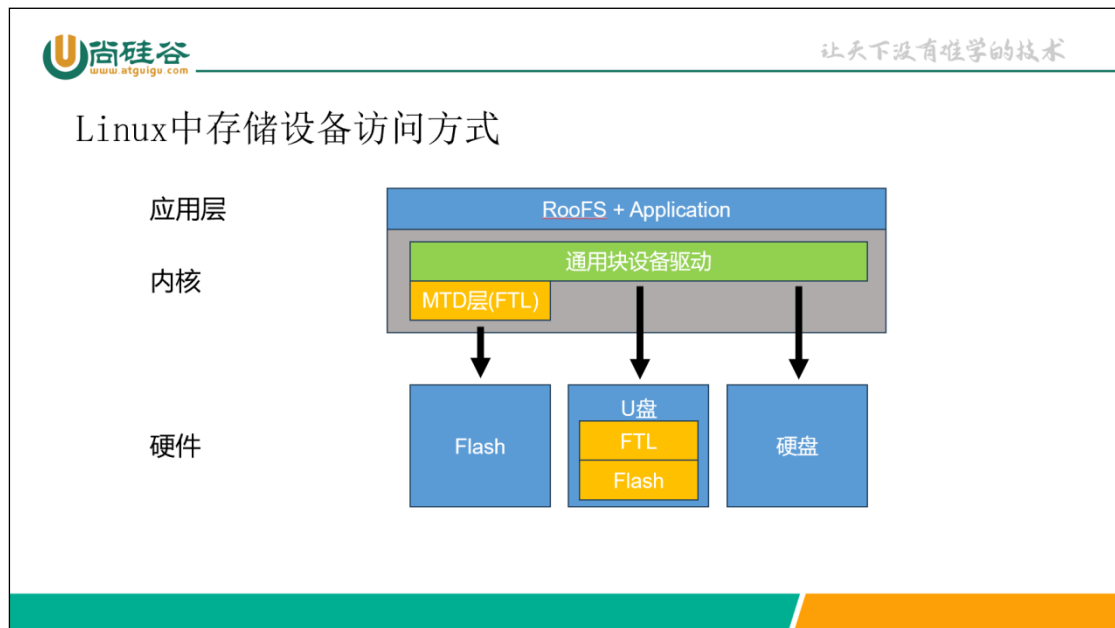
之后我们需要开启 SPI-Flash 驱动。这个驱动分为两个部分：

- (1) 设置默认的启动脚本
- (2) 开启 SOC 芯片的 SPI 驱动支持
- (3) 开启 MTD 驱动支持

第一个选项好理解，我们现在通过芯片硬件 SPI 接口读取 Flash，首先需要芯片通用的 SPI 接口驱动。在这里我们重点解释第二个选项。

MTD (Memory Technology Device)，是 Linux 系统中设备文件系统的一个类别，主要用于闪存的应用，是一种闪存转换层 (Flash Translation Layer, FTL)。创造 MTD 子系统

的主要目的是提供一个介于闪存硬件驱动程序与高阶应用程序之间的抽象层。像 EEPROM, FLASH 等设备都需要通过 MTD 层进行转换, 才能够像使用通用块设备 (例如硬盘) 一样去使用它。严格来说 U 盘或者移动硬盘之类的设备, 底层也是 Flash 颗粒, 但是这类设备在硬件层面就集成了闪存转换层, 对外暴露的接口就是标准的块设备, 所以在驱动层面和裸 Flash 芯片使用的驱动不太一样。下面的图片说明了三者之间的区别。



通过下面的配置开启 SPI 驱动和 MTD 层驱动

```
// 启用 SPL 阶段 Flash 支持
ARM architecture --->
[*] Support for SPI Flash on Allwinner SoCs in SPL
// 设置启动选项
Boot options --->
[*] Enable boot arguments
(console=ttyS0,115200 panic=5 rootwait root=/dev/mtdblock3 rw
rootfstype=squashfs init=/sbin/preinit)
[*] Enable a default value for bootcmd
(sf probe 0; sf read 0x41800000 0x80000 0x8000; sf read 0x41000000
0x88000 0x6F8000; bootz 0x41000000 - 0x41800000;) bootcmd value
Device Tree Control --->
// 启用我们自己的设备树
(sun8i-v3s-atguigupi) Default Device Tree for DT control
// 进入驱动页面
Device Drivers --->
// 开启 SPI 驱动, 会根据 SOC 型号自动选择合适驱动
[*] SPI Support
[*] SPI memory extension
// 进入 MTD 驱动页面。
MTD Support --->
// 开启 MTD 驱动层支持
[*] Enable MTD layer
// 开启标准 MTD 驱动接口
```

```

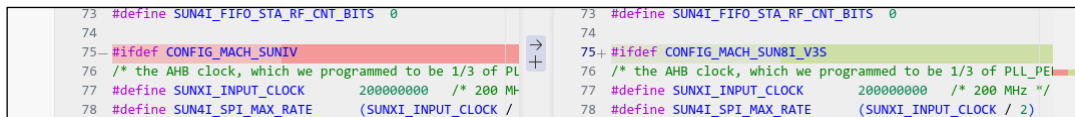
[*] Enable Driver Model for MTD drivers
// 开启 MTD 对于 SPI Flash 的支持
SPI Flash Support --->
[*] SPI Flash Core Interface support
[*] SPI Flash bootdev support
// 开启 SPI Flash 的 bank 切换支持, 大于 16M 的 Flash 都需要开启
[*] SPI flash Bank/Extended address register support
// 我们使用 winbond 的 Flash 芯片, 选中驱动支持
[*] Winbond SPI flash support

```

4.4 修改源码提高 Flash 读取速度

u-boot 中 v3s 的 spi 总线驱动是一个较老的版本, 如果按照默认代码编译, spi 时钟会被限制到 1MHz, 我们可以进行一点自定义修改将其提高到 100MHz。

将 drivers/spi/spi-sunxi.c 的 75 行 CONFIG_MACH_SUNIV 修改为 CONFIG_MACH_SUN8I_V3S, 如图所示:



```

73 #define SUN4I_FIFO_STA_RF_CNT_BITS 0
74
75 #ifdef CONFIG_MACH_SUNIV
76 /* the AHB clock, which we programmed to be 1/3 of PL
77 #define SUNXI_INPUT_CLOCK 200000000 /* 200 M-
78 #define SUN4I_SPI_MAX_RATE (SUNXI_INPUT_CLOCK /
73 #define SUN4I_FIFO_STA_RF_CNT_BITS 0
74
75+ #ifdef CONFIG_MACH_SUN8I_V3S
76 /* the AHB clock, which we programmed to be 1/3 of PLL_PEL
77 #define SUNXI_INPUT_CLOCK 200000000 /* 200 MHz */
78 #define SUN4I_SPI_MAX_RATE (SUNXI_INPUT_CLOCK / 2)

```

4.5 编译 u-boot

按照上述配置 menuconfig 后, 保存退出, 编译 u-boot:

```

atguigu@ubuntu:~/atguigupi/u-boot$ ARCH=arm
CROSS_COMPILE=/home/atguigu/atguigupi/toolchain/bin/arm-linux-
gnueabihf- make -j$(nproc)

```

此时终端会展示编译过程, 最终看到如下反馈, 代表编译成功

```

LDS      spl/u-boot-spl.lds
LD        spl/u-boot-spl
OBJCOPY  spl/u-boot-spl-nodtb.bin
COPY     spl/u-boot-spl.bin
SYM      spl/u-boot-spl.sym
MKIMAGE  spl/sunxi-spl.bin
MKIMAGE  u-boot.img
COPY     u-boot.dtb
MKIMAGE  u-boot-dtb.img
BINMAN   .binman_stamp
OFCHK    .config

```

此时查看 u-boot 目录, 可以看到 u-boot-sunxi-with-spl.bin, 这就是我们最终需要的 u-boot

```

atguigu@ubuntu:~/atguigupi/u-boot$ ll u-boot-sunxi-with-spl.bin
-rw-rw-r-- 1 atguigu atguigu 416368 1月 19 16:07 u-boot-sunxi-with-
spl.bin

```

将文件拷贝到输出目录

```

atguigu@ubuntu:~/atguigupi/u-boot$ mv u-boot-sunxi-with-
spl.bin ../output

```

第 5 章 Linux 内核移植

5.1 下载 Linux 源码

1) 下载源码 tar 包

Linux 源码下载网站: <https://www.kernel.org/>, 选择 longterm 最新的源码下载。

Protocol	Location	Latest Release	
HTTP	https://www.kernel.org/pub/	6.7.1	
GIT	https://git.kernel.org/		
RSYNC	rsync://rsync.kernel.org/pub/		
mainline:	6.8-rc1	2024-01-21	[tarball] [patch] [view diff] [browse]
stable:	6.7.1	2024-01-20	[tarball] [pgp] [patch] [view diff] [browse] [changelog]
stable:	6.6.13	2024-01-20	[tarball] [pgp] [patch] [inc. patch] [view diff] [browse] [changelog]
longterm:	6.1.74	2024-01-20	[tarball] [pgp] [patch] [inc. patch] [view diff] [browse] [changelog]
longterm:	5.15.147	2024-01-15	[tarball] [pgp] [patch] [inc. patch] [view diff] [browse] [changelog]
longterm:	5.10.208	2024-01-15	[tarball] [pgp] [patch] [inc. patch] [view diff] [browse] [changelog]
longterm:	5.4.267	2024-01-15	[tarball] [pgp] [patch] [inc. patch] [view diff] [browse] [changelog]
longterm:	4.19.305	2024-01-15	[tarball] [pgp] [patch] [inc. patch] [view diff] [browse] [changelog]
longterm:	4.14.336 [EOL]	2024-01-10	[tarball] [pgp] [patch] [inc. patch] [view diff] [browse] [changelog]
linux-next:	next-20240122	2024-01-22	[browse]

或者使用资料里面已经附带的 Linux 版本。

2) 解压到 Ubuntu 并重命名

与 u-boot 源码处理方法类似, 再 Ubuntu 中将源码解压, 并重命名为 Linux

```
atguigu@ubuntu:~/atguigupi$ tar -xf linux-6.6.25.tar.xz -C ~/atguigupi
atguigu@ubuntu:~/atguigupi$ mv linux-6.6.25 linux
```

5.2 添加设备树

5.2.1 创建设备树文件

对于内核的设备树文件, 我们仍然从荔枝派 Zero 的设备树文件上进行修改。

```
atguigu@ubuntu:~/atguigupi$ cd linux/arch/arm/boot/dts/allwinner
atguigu@ubuntu:~/atguigupi/linux/arch/arm/boot/dts/allwinner $ touch
sun8i-v3s-atguigupi.dts
```

原来的设备树文件我们不在多做解释, 这里粘贴我们自己的设备树文件:

```
/dts-v1/;
#include "sun8i-v3s.dtsi"
#include "sunxi-common-regulators.dtsi"

/ {
    // 改设备名字
    model = "Atguigu Pi";

    compatible = "atguigu,atguigupi", "allwinner,sun8i-v3s";

    aliases {
        serial0 = &uart0;
```



```
// 给以太网取别名
ethernet0 = &emac;
// 给 SPI 起别名
spi0 = &spi0;
};

chosen {
    stdout-path = "serial0:115200n8";
};
};

&mmc0 {
    broken-cd;
    bus-width = <4>;
    vmmc-supply = <&reg_vcc3v3>;
    status = "okay";
};

&uart0 {
    pinctrl-0 = <&uart0_pb_pins>;
    pinctrl-names = "default";
    status = "okay";
};

// 开启以太网
&emac {
    allwinner,leds-active-low;
    status = "okay";
};

// 开启 SPI 和 FLASH
&spi0 {
    status = "okay";
    w25q256:w25q256@0 {
        compatible = "winbond,w25q256", "jedec,spi-nor";
        reg = <0x0>;
        spi-max-frequency = <50000000>;
        #address-cells = <1>;
        #size-cells = <1>;
        // 由于 Flash 没有分区表，需要在设备树文件中声明 flash 分区
        // 这里总共分了 5 个区

        partitions {
            compatible = "fixed-partitions";
```

```
#address-cells = <1>;
#size-cells = <1>;

partition-uboot {
    reg = <0x0 0x80000>;
    read-only;
};

partition-dtb {
    reg = <0x80000 0x8000>;
    read-only;
};

partition-kernel {
    reg = <0x88000 0x6F8000>;
    read-only;
};

partition-rootfs {
    reg = <0x780000 0x1080000>;
    read-only;
};
partition-data {
    reg = <0x1800000 0x800000>;
};
};
};
```

5.2.2 修改设备树 makefile

我们新添加的设备树文件由于没有在 makefile 中注册，并不会被内核编译。我们还需要在 makefile 中注册我们的设备树文件。用 VSCode 打开 arch/arm/boot/dts/allwinner/Makefile 文件，在 323 行添加如下内容：

321	sun8i-v3s-licheepi-zero.dtb \
322	sun8i-v3s-licheepi-zero-dock.dtb \
323	sun8i-v3s-atguigupi.dtb \
324	sun8i-v40-bananapi-m2-berry.dtb
325	dtb-\$(CONFIG_MACH_SUN9I) += \

```
sun8i-v3s-atguigupi.dtb \
```

5.3 修改 menuconfig

对 Linux 内核进行编译配置：

```
atguigu@ubuntu:~/atguigupi/linux/arch/arm/boot/dts$ cd
~/atguigupi/linux
atguigu@ubuntu:~/atguigupi/linux$ export ARCH=arm
atguigu@ubuntu:~/atguigupi/linux$ export CROSS_COMPILE=arm-linux-
gnueabi-
atguigu@ubuntu:~/atguigupi/linux$ make sunxi_defconfig
atguigu@ubuntu:~/atguigupi/linux$ make menuconfig
```

开启 menuconfig 后，进行我们需要的自定义修改，修改的部分主要包括下面的内容：

- 开启 mtd 层驱动支持
- 开启 mtd spi flash 支持
- 开启 squashfs、jffs2、overlayfs 文件系统支持
- 开启 ipv6 支持（防止 ipv6 报错）

mtd 和 mtd spi 的作用，在上一节 u-boot 驱动中已经解释过了；ipv6 支持其实是为了防止在 Linux 使用中的一个 ipv6 报错，不开也不影响实际使用；这里解释一下三种文件系统支持：

1) squashfs

squashfs 是一种压缩的只读文件系统，会将其上的所有文件打包成一个只读的 rom。我们将来打算采用 squashfs 作为根文件系统（rootfs），这样做的好处有两个：

- squashfs 会采用 zlib 压缩算法，文件系统整体较小，在嵌入式应用非常合适；
- flash 擦写次数有限，并且在擦写过程中可能造成数据损坏。采用 squashfs 做根文件系统，可以保证存储根文件系统的区域是安全的，增加系统的安全性。

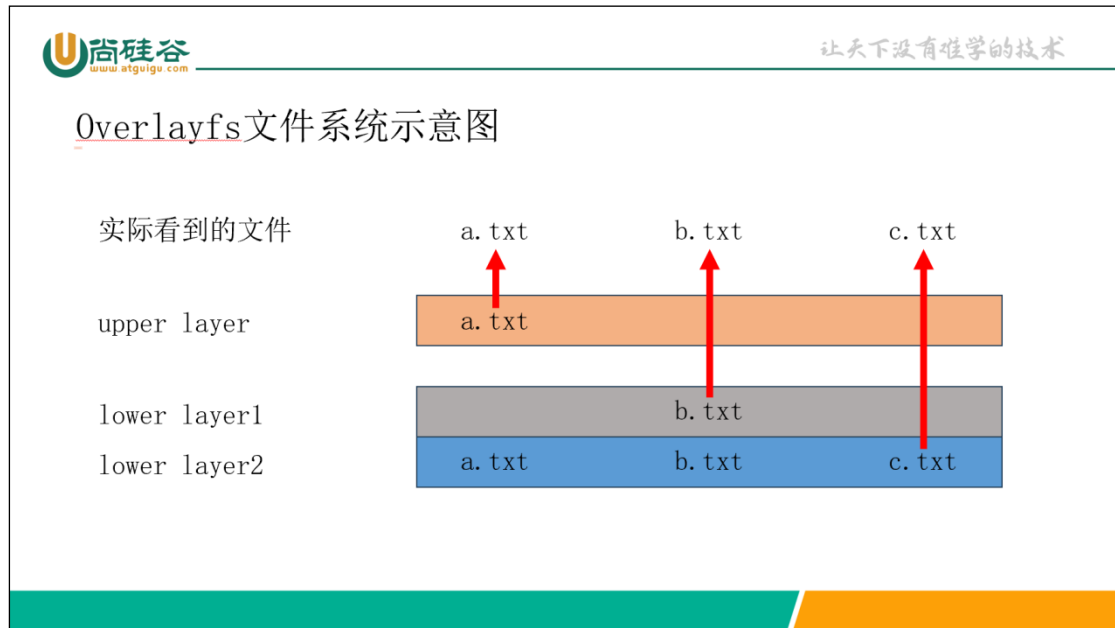
但是，如果只采用 squashfs 作为文件系统，整个文件系统就是只读的，实际使用中这样做肯定不行。

2) jffs2

jffs2 是一种专门为 mtd spi flash 设备设计的文件系统。由于使用 flash 作为底层存储，在制作文件系统时，我们不能采用常见的 ext4/xfs 文件系统，而是需要使用专门为 flash 制作的文件系统。这个文件系统不支持 flash 4k 扇区擦写功能，所以还要在 spi flash 驱动中将这个功能关掉。

3) overlayfs

overlayfs 是一种多层文件系统，可以将不同的文件系统以层叠的形式合而为一挂载在同一个目录。它由一个 upper、任意层 lower 文件系统共同构成，只有 upper 层可写，所有的 lower 层都是只读的，对所有文件的修改最终都保存在 upper 层。其示意图如下：



最终按照如下说明修改:

```
[*] Networking support --->
Networking options --->
// 选中 ipv6 协议
<*> The IPv6 protocol --->
Device Drivers --->
// 选中 mtd 支持并进入子菜单
<*> Memory Technology Device (MTD) support --->
// 选中 spi nor flash 支持并进入子菜单
<*> Caching block device access to MTD devices
<*> SPI NOR device support --->
// 取消 4k sector 擦写 (和 jffs2 冲突)
[ ] Use small 4096 B erase sectors
[*] SPI support --->
// 取消 A10 处理器的 SPI 支持 (只留下下面的 A31)
< > Allwinner A10 SoCs SPI controller
File systems --->
// 开启 overlayfs 支持
<*> Overlay filesystem support
[*] Miscellaneous filesystems --->
// 开启 jffs2 支持
<*> Journaled Flash File System v2 (JFFS2) support
[*] JFFS2 XATTR support
// 开启 squashfs 支持
<*> SquashFS 4.0 - Squashed file system support
[*] Squashfs XATTR support
```

5.4 编译 Linux 内核

1) 编译内核:

```
atguigu@ubuntu:~/atguigupi/linux$ make -j$(nproc) zImage
```

看到如下信息表示编译成功:

```
OBJCOPY arch/arm/boot/zImage
```

```
Kernel: arch/arm/boot/zImage is ready
```

2) 编译设备树:

```
atguigu@ubuntu:~/atguigupi/linux$ make -j$(nproc) dtbs
```

看到如下信息表示编译成功:

```
DTC      arch/arm/boot/dts/sun8i-v3s-atguigupi.dtb
```

3) 将编译的结果拷贝到 output 目录:

```
atguigu@ubuntu:~/atguigupi/linux$ cp arch/arm/boot/zImage ../output
atguigu@ubuntu:~/atguigupi/linux$ cp arch/arm/boot/dts/sun8i-v3s-atguigupi.dtb ../output
```

第 6 章 根文件系统移植

6.1 下载 buildroot 源码

buildroot 是一个可以方便构建根文件系统 (rootfs) 的项目, 下载地址:

<https://buildroot.org/download.html>, 选择 buildroot-2023.02.9.tar.gz 下载即可。下载之后我们

还是将源码解压在~/atguigupi 下面:

```
atguigu@ubuntu:~/atguigupi$ wget
https://buildroot.org/downloads/buildroot-2023.02.9.tar.gz
atguigu@ubuntu:~/atguigupi$ tar -xf buildroot-2023.02.9.tar.gz
atguigu@ubuntu:~/atguigupi$ mv buildroot-2023.02.9 buildroot
```

6.2 修改 menuconfig

```
atguigu@ubuntu:~/atguigupi$ cd buildroot
atguigu@ubuntu:~/atguigupi/buildroot$ make menuconfig
```

按照如下内容修改:

```
Target options --->
// arm 架构, A7 指令集
Target Architecture (ARM (little endian)) --->
Target Architecture Variant (cortex-A7) --->
Toolchain --->
// 使用外部工具链, 并且是自定义工具链
Toolchain type (External toolchain) --->
Toolchain (Custom toolchain) --->
// 不用 buildroot 下载, 我们提前准备好工具链
Toolchain origin (Pre-installed toolchain) --->
// 工具链地址, 只需要写到 toolchain 一级
(/home/atguigu/atguigupi/toolchain) Toolchain path
// 工具链固定前缀
/arm-linux-gnueabihf) Toolchain prefix
// 工具链版本
External toolchain gcc version (12.x) --->
// 确认是 6.1 和 header, 这个要和我们的内核版本对应
External toolchain kernel headers series (6.1.x or later) --->
// 使用 glibc
External toolchain C library (glibc) --->
// 开启 C++ 和 OpenMP 支持
[*] Toolchain has C++ support?
```

```

[*] Toolchain has OpenMP support?
System configuration --->
// 设置 hostname 和欢迎信息
(atguigupi) System hostname
(Welcome to Atguigu Pi) System banner
// 设置/bin、/sbin、/lib 三个目录为软连接
[*] Use symlinks to /usr for /bin, /sbin and /lib
// 设置初始 root 密码
(1) Root password
// 设置初始启用的网卡
(eth0) Network interface to configure through DHCP
// 设置语言支持
(C zh_CN.UTF-8) Locales to keep
// 安装时区
[*] Install timezone info
(Asia/Shanghai) default local time
// 设置自定义 rootfs 内容
(rootfs-overlay) Root filesystem overlay directories
Target packages --->
Hardware handling --->
// 硬件随机数生成器，增加系统随机数性能
[*] rng-tools
Libraries --->
Crypto --->
// 使用 libressl 库（替换 openssl）
ssl library (libressl) --->
Networking applications --->
// 使用 dropbear ssh 服务端
[*] dropbear
[*] disable reverse DNS lookups
Filesystem images --->
// 将结果生成 squashfs 镜像
[*] squashfs root filesystem

```

6.3 自定义 rootfs 内容

1) 创建目录结构

在上面的配置中，我们配置以下选项

```
(rootfs-overlay) Root filesystem overlay directories
```

这个选项允许我们将 rootfs-overlay 目录下面的内容添加到最终的镜像里面，以便我们对最终的 rootfs 做一些自定义修改。首先我们新建 rootfs-overlay 目录：

```
atguigu@ubuntu:~/atguigupi/buildroot$ mkdir -p rootfs-overlay
```

然后在 rootfs-overlay 目录中新建以下子目录：

```

atguigu@ubuntu:~/atguigupi/buildroot$ cd rootfs-overlay
atguigu@ubuntu:~/atguigupi/buildroot/rootfs-overlay$ mkdir -p chroot
rom overlay root etc/dropbear usr/sbin
atguigu@ubuntu:~/atguigupi/buildroot/rootfs-overlay$ ll

```

结果如下：

```

总计 28
drwxrwxr-x 7 skiinder skiinder 4096 1月 30 16:29 ./

```

```
drwxr-xr-x 17 skiinder skiinder 4096 1月 30 16:29 ../
drwxrwxr-x 2 skiinder skiinder 4096 1月 30 16:29 chroot/
drwxrwxr-x 2 skiinder skiinder 4096 1月 30 16:29 etc/
drwxrwxr-x 2 skiinder skiinder 4096 1月 30 16:29 overlay/
drwxrwxr-x 2 skiinder skiinder 4096 1月 30 16:29 rom/
drwxrwxr-x 2 skiinder skiinder 4096 1月 30 16:29 root/
```

2) 生成 preinit 文件

由于我们的文件系统是 squashfs + jffs2 的双层文件系统，我们需要在内核引导的过程中完成这两个文件系统的挂载；然而内核刚刚启动时，我们只能挂载 squashfs 镜像作为 rootfs，这就需要在系统真正初始化之前完成文件系统的二次挂载。

Linux 在内核引导结束以后，第一个用户态进程就是/sbin/init。这个进程的 PID 必须为 1，它负责执行用户态最基本的初始化，拉起其他用户态进程（登录界面、ssh 服务端、DHCP 客户端，Shell 等），我们要在/sbin/init 执行之前插入一些自己的代码（挂载 overlayfs，切换 overlayfs 作为 rootfs 后再执行/sbin/init），这可以通过在引导过程中指定特定初始化脚本实现。现在我们首先来实现这个初始化脚本：

使用 vscode 在 rootfs-overlay/usr/sbin 目录中新建 preinit 脚本，内容如下

```
#!/bin/sh

# mount the new rootfs
mount -r -t squashfs /dev/mtdblock3 /rom
mount -t jffs2 /dev/mtdblock4 /overlay
mount -n -t overlay overlayfs:/overlay -o
lowerdir=/rom,upperdir=/overlay/data,workdir=/overlay/work /chroot

# mount /dev and /sys for chroot environment
# /dev/pts, /dev/shm, /proc, /tmp and /run will be mounted by inittab
from fstab
mount -t sysfs sysfs /sys
mount --rbind /sys /chroot/sys/
mount --rbind /dev /chroot/dev/

exec chroot /chroot /sbin/init
```

保存，在终端中给文件增加执行权限

```
atguigu@ubuntu:~/atguigupi/buildroot/rootfs-overlay$ chmod +x
usr/sbin/preinit
```

3) 生成 profile 文件

我们还需要在 root 用户登录时，执行一些基本 shell 环境定义，让我们的 shell 环境使用更顺手。可以通过在 rootfs-overlay/root 目录下，新建 profile 文件实现。

在 vscode 中新建 rootfs-overlay/root/.profile, 内容如下:

```
# ~/.profile: executed by Bourne-compatible login shells.
HISTCONTROL=ignoredups:ignorespace
HISTSIZE=1000
HISTFILESIZE=2000

if [ -z "$debian_chroot" ] && [ -r /etc/debian_chroot ]; then
    debian_chroot=$(cat /etc/debian_chroot)
fi

if [ "$color_prompt" = yes ]; then
    PS1='${debian_chroot:+($debian_chroot)}\[\033[01;32m\]\u@\h\[\033[00m\]:\[\033[01;34m\]\w\[\033[00m\]\$ '
else
    PS1='${debian_chroot:+($debian_chroot)}\u@\h:\w\$ '
fi
unset color_prompt force_color_prompt

# If this is an xterm set the title to user@host:dir
case "$TERM" in
xterm*|rxvt*)
    PS1="\[\e]0;${debian_chroot:+($debian_chroot)}\u@\h: \w\a\]$PS1"
    ;;
*)
    ;;
esac

# enable color support of ls and also add handy aliases
if [ -x /usr/bin/dircolors ]; then
    test -r ~/.dircolors && eval "$(dircolors -b ~/.dircolors)" || eval "$(dircolors -b)"
    alias ls='ls --color=auto'
    #alias dir='dir --color=auto'
    #alias vdir='vdir --color=auto'

    alias grep='grep --color=auto'
    alias fgrep='fgrep --color=auto'
    alias egrep='egrep --color=auto'
fi

# some more ls aliases
alias ll='ls -lF'
alias la='ls -A'
```



```
alias l='ls -CF'

if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi

mesg n 2> /dev/null || true
```

最终的目录结构:

```
rootfs-overlay
├── chroot
├── etc
│   └── dropbear
├── overlay
├── rom
├── root
│   └── .profile
├── usr
│   └── sbin
│       └── preinit
```

6.4 生成 rootfs 镜像

执行编译指令，即可生成 rootfs 镜像。buildroot 需要从工具链开始编译，整个执行过程比较漫长，大家要耐心等待

```
atguigu@ubuntu:~/atguigupi/buildroot$ make -j$(nproc)
```

当收到如下反馈时代表 rootfs 镜像生成成功了

```
Parallel mksquashfs: Using 17 processors
Creating 4.0 filesystem on
/home/skiinder/atguigupi/buildroot/output/images/rootfs.squashfs, block
size 65536.
[=====] 197/197 100%
```

将生成的镜像拷贝到输出目录

```
atguigu@ubuntu:~/atguigupi/buildroot$ mv
output/images/rootfs.squashfs ../output
```

6.5 制作 jffs2 镜像

最后我们需要制作一个作为 overlayfs 的 upper 层的 jffs2 镜像。

```
atguigu@ubuntu:~/atguigupi/buildroot$ mkdir -p fateroot/data
fateroot/work
atguigu@ubuntu:~/atguigupi/buildroot$ fakeroot mkfs.jffs2 -s 0x100 -e
0x10000 --pad=0x1000000 -o ../output/data.jffs2 -r fakeroot
atguigu@ubuntu:~/atguigupi/buildroot$ rm -rf fakeroot
```

第 7 章 制作 Flash 镜像并烧录

7.1 制作 Flash 镜像

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

当所有工作准备就绪，进入 output 目录：

```
atguigu@ubuntu:~/atguigupi/buildroot$ cd ../output
atguigu@ubuntu:~/atguigupi/output$ ll
```

看到如下内容：

```
-rw-r--r-- 1 skiinder skiinder 16777216 1月 30 18:50 data.jffs2
-rw-r--r-- 1 skiinder skiinder 3334144 1月 30 17:30 rootfs.squashfs
-rw-rw-r-- 1 skiinder skiinder 12246 1月 29 18:47 sun8i-v3s-
atguigupi.dtb
-rw-rw-r-- 1 skiinder skiinder 437696 1月 29 18:57 u-boot-sunxi-with-
spl.bin
-rwxrwxr-x 1 skiinder skiinder 5542656 1月 30 17:46 zImage
```

在 vscode 中新建 output/flash.sh，内容如下：

```
#!/bin/env bash
IMG_FILE="flash-image"
UBOOT_FILE="u-boot-sunxi-with-spl.bin"
DTB_FILE="sun8i-v3s-atguigupi.dtb"
KERNEL_FILE="zImage"
ROOTFS_FILE="rootfs.squashfs"
OVERLAYFS_FILE="data.jffs2"

# 制作镜像
function make_image() {
    set -e
    # 生成 32M 空文件
    dd if=/dev/zero of=$IMG_FILE bs=1M count=32
    # offset=0 写入 uboot
    dd if=$UBOOT_FILE of=$IMG_FILE bs=1K seek=0 conv=notrunc
    # offset=512K 写入 dtb
    dd if=$DTB_FILE of=$IMG_FILE bs=1K seek=512 conv=notrunc
    # offset=544K 写入内核
    dd if=$KERNEL_FILE of=$IMG_FILE bs=1K seek=544 conv=notrunc
    # offset=7.5M 写入 rootfs
    dd if=$ROOTFS_FILE of=$IMG_FILE bs=1K seek=7680 conv=notrunc
    # offset=24M 写入 data.jffs2
    dd if=$OVERLAYFS_FILE of=$IMG_FILE bs=1K seek=24576 conv=notrunc
}

# 检查 flash 是否存在
function ckeck_flash() {
    local RESULT="$(sudo ./sunxi-fel spiflash-info)"
    test "$RESULT" = "Manufacturer: Winbond (EFh), model: 40h, size:
33554432 bytes."
}
```

```
function flash_image() {
    make_image
    echo "镜像制作完成, 开始刷写镜像"
    ckeck_flash && sudo ./sunxi-fel -p spiflash-write 0 $IMG_FILE ||
echo "没有找到 Flash"
}

function flash_uboot() {
    echo "开始刷写 uboot"
    ckeck_flash && sudo ./sunxi-fel -p spiflash-write 0 $UBOOT_FILE ||
echo "没有找到 Flash"
}

function flash_dtb() {
    echo "开始刷写 dtb"
    ckeck_flash && sudo ./sunxi-fel -p spiflash-write 524288 $DTB_FILE
|| echo "没有找到 Flash"
}

function flash_kernel() {
    echo "开始刷写 kernel"
    ckeck_flash && sudo ./sunxi-fel -p spiflash-write 557056
$KERNEL_FILE || echo "没有找到 Flash"
}

function flash_rootfs() {
    echo "开始刷写 rootfs"
    ckeck_flash && sudo ./sunxi-fel -p spiflash-write 7864320
$ROOTFS_FILE || echo "没有找到 Flash"
}

function flash_data() {
    echo "开始刷写 data"
    ckeck_flash && sudo ./sunxi-fel -p spiflash-write 25165824
$OVERLAYFS_FILE || echo "没有找到 Flash"
}

function print_usage() {
    echo "flash image|uboot|dtb|kernel|rootfs|data|all"
}

echo "测试 sudo 权限"
sudo echo "成功" || echo "无法执行 sudo"
```

```
for CMD in "$@"; do
    case $CMD in
        "uboot" | "dtb" | "kernel" | "rootfs" | "data") ;;
        "image")
            flash_image
            exit 0
            ;;
        "all")
            flash_uboot
            flash_dtb
            flash_kernel
            flash_rootfs
            flash_data
            exit 0
            ;;
        *)
            print_usage
            exit 1
            ;;
    esac
done

for CMD in "$@"; do
    flash_${CMD}
done
```

保存备用

7.2 烧录 Flash 镜像

烧录 Flash 镜像需要 v3s 进入 fel 模式。根据 v3s 启动顺序，如果 flash 和 sdcard 全部启动失败，就会进入 fel 模式。所以我们可以按照如下步骤烧录：

- (1) 要退出 sd 卡
- (2) 将 spi 的 ce 引脚短接到 gnd
- (3) 连接电脑和开发板的 otg 口
- (4) 等 3 秒钟，断开 ce 和 gnd 引脚

此时开发板应该已经进入了 fel 模式，我们可以将开发板连接到虚拟机来验证一下。

选中 ubuntu 虚拟机，选择虚拟机菜单中的“虚拟机——可移动设备——Allwinner RS2——连接”，再将资料包中的 sunxi-fel 拷贝到 output 文件夹，增加执行权限，执行下面的命令刷机：

```
atguigu@ubuntu:~/atguigupi/output$ ./flash.sh all
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

如果能看到刷机提示，即表示开始刷机，等待所有进度条走满，代表刷机结束。

```
100% [=====] 33554 kB, 87.0 kB/s
```

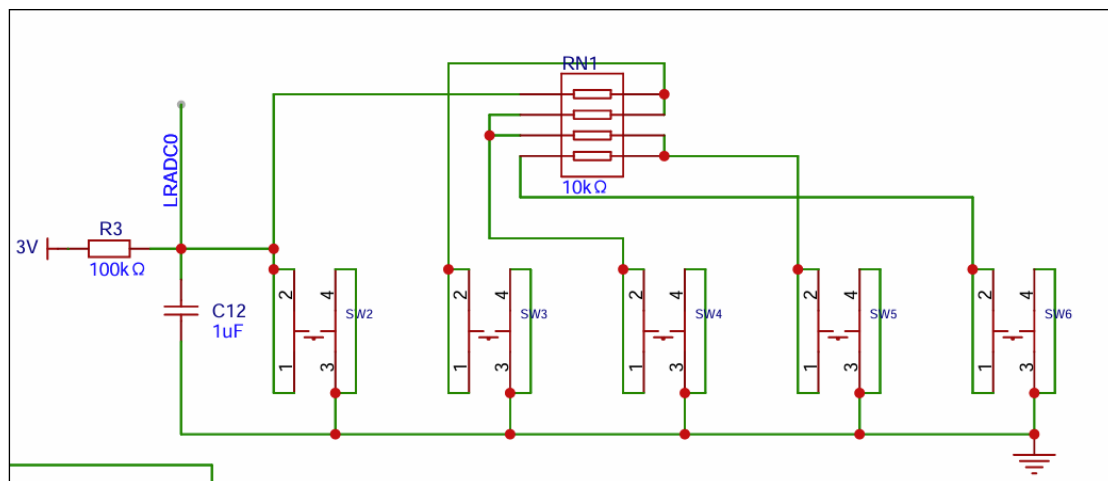
之后给开发板上电，即可启动 Linux 了。

第 8 章 驱动移植

现在我们的开发板已经可以跑 Linux 了，但是我们的开发板上还有很多外设。如果想让这些外设 in Linux 中发挥作用，需要我们为各种外设编写驱动和设备树。其实 Linux 主线上常见的外设驱动代码都是现成的，我们只需要为其编写合适的设备树即可开启该设备，只有个别驱动需要我们适配 Linux 6.x 的 API

8.1 按键驱动

8.1.1 原理图



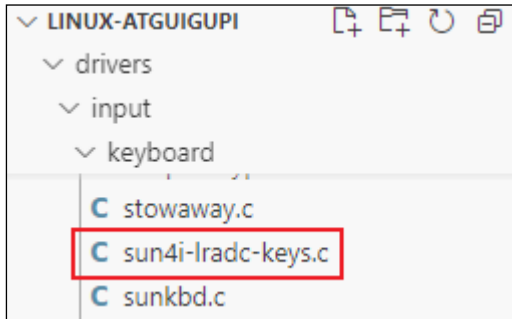
从原理图上可以看到全志 v3s 有一个模拟量输入引脚，通过不同按键按下而输入电压不一样从而区分按键不同。查询手册可知，v3s 这个引脚为 6bit 精度，也就是说有 0-63 共有 64 挡位。

8.1.2 驱动移植

查看 Linux 内核源码，可以看到这个独立按键的驱动已经有人开发过了，所以我们不用从零开始写代码，主要的工作有以下两个：

- 适配该驱动的设备树，并在内核编译时打开该驱动的编译选项
- 为该驱动校准电压

首先我们需要查看一下是否已经存在驱动文件。Linux 内核所有的驱动都在 drivers 目录下，而按键属于输入设备中的键盘输入，我们进入 drivers/input/keyboard 目录下，可以看到 sun4i-lradc-keys.c 文件，这个文件就是我们的按键驱动。



有了这个按键驱动以后，我们就可以为这个驱动写一个设备树节点。编写设备树需要对照驱动提供的设备树说明。以这个按键驱动为例，其设备树说明为 Documentation/devicetree/bindings/input/allwinner,sun4i-a10-lradc-keys.yaml。在这个说明文件的末尾，我们可以找到一个设备树节点的例子，对照这个例子，我们可以在文档中找到每个词条的说明：

```
// 节点名称为 lradc，地址为 1c22800
lradc: lradc@1c22800 {
    // 兼容属性
    compatible = "allwinner,sun4i-a10-lradc-keys";
    // 寄存器地址和范围
    reg = <0x01c22800 0x100>;
    // 中断号
    interrupts = <31>;
    // 参考电压供应
    vref-supply = <&reg_vcc3v0>;
    // 名为 button-191 的子节点
    button-191 {
        // 标签为"Volume Up"
        label = "Volume Up";
        // Linux 下的键码为 115
        linux,code = <115>;
        // 通道为 0
        channel = <0>;
        // 电压值
        voltage = <191274>;
    };
    // 名为 button-392 的子节点
    button-392 {
        // 标签为"Volume Down"
        label = "Volume Down";
        // Linux 下的键码为 114
        linux,code = <114>;
        // 通道为 0
        channel = <0>;
    };
};
```

```
// 电压值
voltage = <392644>;
};
};
```

这里面我们需要重点关注的为：寄存器地址，中断号、按键电压、按键号。如果从 0 开始写设备树，这些内容都需要我们查询手册，调试按键等等，不过我们通过查看 sun8i-v3s.dtsi 源码可以看到，lradc 节点的寄存器和范围已经声明好了：

```
460 lradc: lradc@1c22800 {
461     compatible = "allwinner,sun4i-a10-lradc-keys";
462     reg = <0x01c22800 0x400>;
463     interrupts = <GIC_SPI 30 IRQ_TYPE_LEVEL_HIGH>;
464     status = "disabled";
465 };
```

这样我们在自己的设备树中就可以直接引用这个节点，并添加自己的按键节点：

```
#include <dt-bindings/input/linux-event-codes.h>
```

```
&lradc {
    vref-supply = <&reg_vcc3v0>;
    status = "okay";
    button-0 {
        label = "Left";
        linux,code = <KEY_LEFT>;
        channel = <0>;
        voltage = <0>;
    };
    button-273 {
        label = "Right";
        linux,code = <KEY_RIGHT>;
        channel = <0>;
        voltage = <412698>;
    };
    button-500 {
        label = "Up";
        linux,code = <KEY_UP>;
        channel = <0>;
        voltage = <825396>;
    };
    button-692 {
        label = "Down";
        linux,code = <KEY_DOWN>;
        channel = <0>;
        voltage = <1111111>;
    };
};
```

```
button-857 {
    label = "OK";
    linux,code = <KEY_OK>;
    channel = <0>;
    voltage = <1269841>;
};
};
```

这些按键的电压不一定是准确的，我们需要在下一步实际测试中进行调试。

8.1.3 按键驱动调试

如果我们要在 Linux 中调试按键驱动，需要用到内核调试输出辅助我们。打开按键驱动源码，按照如下修改

```
// 在 63 行添加如下宏定义
#define DEBUG_INPUT_KEY
// 在 138 行添加如下日志输出。每当我们按下按键触发中断，就会输出相关的按键信息
#ifdef DEBUG_INPUT_KEY
    printk("Debug: volatile: %u V, adc: %d,ref volatage: %d button
pressed: %d\n", voltage, val, lradc->vref, keycode);
#endif
```

修改完驱动和设备树，我们重新编译内核和设备树文件，并刷写到开发板，即可进行按键调试。当我们按下下一个按键后，系统日志会输出相应的信息，根据信息我们重新校准按键电压即可。

重新编译设备树和内核并刷入，即可测试驱动。

8.2 第二个串口

我们将来的项目会用到第二个串口。串口的设备树和驱动，之前的设备树文件中已经有了，我们直接使用即可。首先查看设备树头文件 sun8i-v3s.dtsi，可以看到串口 2 的声明：

```
512     uart2: serial@1c28800 {
513         compatible = "snps,dw-apb-uart";
514         reg = <0x01c28800 0x400>;
515         interrupts = <GIC_SPI 2 IRQ_TYPE_LEVEL_HIGH>;
516         reg-shift = <2>;
517         reg-io-width = <4>;
518         clocks = <&ccu CLK_BUS_UART2>;
519         dmas = <&dma 8>, <&dma 8>;
520         dma-names = "tx", "rx";
521         resets = <&ccu RST_BUS_UART2>;
522         pinctrl-0 = <&uart2_pins>;
523         pinctrl-names = "default";
524         status = "disabled";
525     };
```


所以我们在自己的设备树中打开并起个别名就可以使用了。

```
aliases {
    serial2 = &uart2;
};

&uart2 {
    status = "okay";
};
```

重新编译设备树并刷入，即可使用第二个串口。

8.3 I2C 总线

在我们的开发板上，总共有两条 I2C 总线，这在设备树头文件中也有声明。不过 I2C1 引脚有复用，如下图所示：

391			<code>/omit-if-no-ref/</code>
392			<code>i2c1_pb_pins: i2c1-pb-pins {</code>
393			<code>pins = "PB8", "PB9";</code>
394			<code>function = "i2c1";</code>
395			<code>};</code>
396			
397			<code>/omit-if-no-ref/</code>
398			<code>i2c1_pe_pins: i2c1-pe-pins {</code>
399			<code>pins = "PE21", "PE22";</code>
400			<code>function = "i2c1";</code>
401			<code>};</code>

我们为了使用这两条 I2C 总线，需要打开状态开关，起别名，并选择复用引脚：

```
aliases {
    i2c0 = &i2c0;
    i2c1 = &i2c1;
};

&i2c0 {
    status = "okay";
};

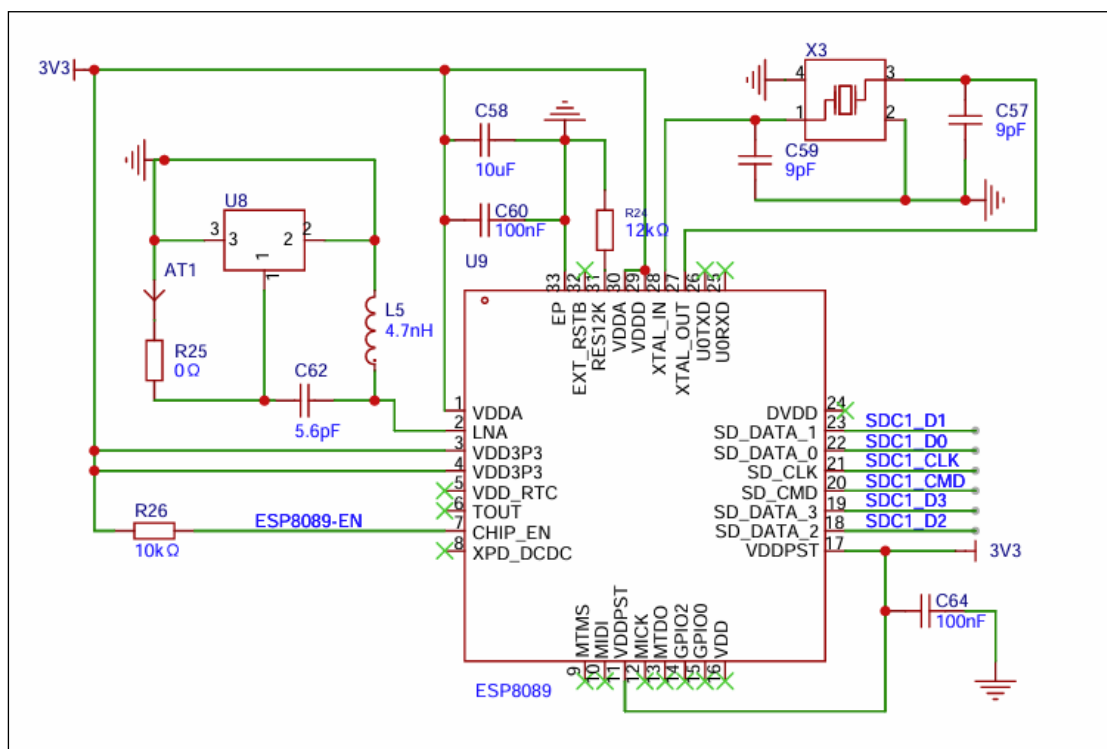
&i2c1 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&i2c1_pe_pins>;
};
```

重新编译设备树并刷入（驱动已经在内核中启用了），查看/dev/i2c0 和/dev/i2c1 两个设备。

8.4 无线网卡

8.4.1 原理图

全志 v3s 本身是没有集成无线网卡的，我们的开发板使用了 ESP8089 作为无线网卡。ESP8089 提供了一个完整且独立的 Wi-Fi 网络解决方案。当作为 Wi-Fi 适配器时，ESP8089 可以与任何基于微控制器的系统配合工作，通过 SPI/SDIO 接口实现无线连接。在我们的开发板上，ESP8089 通过 SDIO1 接口与主控相连，原理图如下：



8.4.2 驱动移植

1) 设备树文件

如果想使用 ESP8089，我们首先需要在设备树中启用 SDIO1。

```
// 开放 mmc1 设备，无线网卡挂在这个接口下面
&mmc1 {
    broken-cd;
    bus-width = <4>;
    vmmc-supply = <&reg_vcc3v3>;
    status = "okay";
};
```

并且为 ESP8089 编写驱动程序。

2) 重新编译内核

无线网卡依赖于 mac80211，我们需要重新编译并刷入内核，开启 mac80211 支持。配置如下：

```
[*] Networking support --->
[*]   Wireless --->
    <*>   cfg80211 - wireless configuration API
    [*]   enable powersave by default
    <*>   Generic IEEE 802.11 Networking Stack (mac80211)
    [*]   Enable mac80211 mesh networking support
    [*]   Select mac80211 debugging features --->
```

3) 驱动程序

ESP8089 网卡的驱动程序是树外驱动，需要单独编译。

```
# 进入 atguigupi 目录，克隆驱动项目
atguigu@ubuntu:~/atguigupi$ git clone
https://github.com/skiinder/esp8089.git
atguigu@ubuntu:~/atguigupi$ cd esp8089
# 切换到 kernel-6.6 分支
atguigu@ubuntu:~/atguigupi/esp8089$ git checkout kernel-6.6
# 执行驱动编译工作
atguigu@ubuntu:~/atguigupi/esp8089$ export ARCH=arm
CROSS_COMPILE=/home/atguigu/atguigupi/toolchain/bin/arm-linux-
gnueabi-
atguigu@ubuntu:~/atguigupi/esp8089$ make -C ../linux/ M=$PWD modules
```

待编译完成，目录中 esp8089.ko 就是驱动文件。

4) 重新构建 rootfs

构建完成的驱动程序，我们可以直接将其打包到 rootfs 中，方便加载。

```
# 首先准备驱动程序
atguigu@ubuntu:~/atguigupi/esp8089$ cd ../buildroot
atguigu@ubuntu:~/atguigupi/buildroot$ LINUX_RELEASE=$(cat ../linux/include/generated/utsrelease.h | grep UTS_RELEASE | cut -d '"' -f 2)
atguigu@ubuntu:~/atguigupi/buildroot$ mkdir -p rootfs-
overlay/usr/lib/modules/$LINUX_RELEASE
atguigu@ubuntu:~/atguigupi/buildroot$ cp ../esp8089/esp8089.ko rootfs-
overlay/usr/lib/modules/$LINUX_RELEASE/
atguigu@ubuntu:~/atguigupi/buildroot$ cat <<'EOF' >rootfs-
overlay/usr/lib/modules/$LINUX_RELEASE/modules.dep
esp8089.ko:
EOF

# 其次准备驱动程序挂载脚本
atguigu@ubuntu:~/atguigupi/buildroot$ cat <<'EOF' >rootfs-
overlay/etc/init.d/S01esp8089
#!/bin/sh

case $1 in
start)
    echo "Installing esp8089: "
    modprobe esp8089 && echo "ok" || echo "fail"
    ;;
stop)
```

```

echo "Uninstalling esp8089: "
modprobe -r esp8089 && echo "ok" || echo "fail"
;;
esac
EOF

atguigu@ubuntu:~/atguigupi/buildroot$ chmod +x rootfs-
overlay/etc/init.d/S01esp8089

# 配置网络 DHCP 连接
atguigu@ubuntu:~/atguigupi/buildroot$ cat <<'EOF' >rootfs-
overlay/etc/network/interfaces
auto lo
iface lo inet loopback

auto eth0
iface eth0 inet dhcp
pre-up /etc/network/nfs_check
wait-delay 15
hostname $(hostname)

auto wlan0
iface wlan0 inet dhcp
pre-up /etc/network/nfs_check
wpa-conf /etc/wpa_supplicant.conf
wait-delay 15
hostname $(hostname)
EOF

```

编辑 buildroot 选项，加如 wpa_supplicant 支持

```

Target packages --->
Networking applications --->
[*] wpa_supplicant --->
    这个分支下面的选项全部选中

```

完成后重新构建 rootfs，并重新刷入新构建的 kernel 和 rootfs，重启即可看到无线网卡。

8.4.3 连接 wifi 网络

刷机并重启后，可以尝试用命令行连接 WiFi 网络：

```

# 添加无线网络
wpa_cli -i wlan0 add_network //添加一个网络连接,会返回<network id>
wpa_cli set_network <network id> ssid '"name"' //ssid 名称
wpa_cli set_network <network id> psk ' "psk" ' //密码
wpa_cli set_network <network id> scan_ssid 1
wpa_cli set_network <network id> priority 1 //优先级

# 保存配置
wpa_cli -i wlan0 save_config

# 连接网络
wpa_cli -i wlan0 select_network <network id> //连接指定的 ssid
wpa_cli -i wlan0 enable_network <network id> //使能指定的 ssid

```

8.5 USB 总线

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

全志 V3s 继承了一个 USB 控制器，我们可以打开该部分的设备树，即可启用 USB 总线。

USB 总线设备树分两个节点，如下图所示：

```

307  ✓      usb_otg: usb@1c19000 {
308          compatible = "allwinner,sun8i-h3-musb";
309          reg = <0x01c19000 0x0400>;
310          clocks = <&ccu CLK_BUS_OTG>;
311          resets = <&ccu RST_BUS_OTG>;
312          interrupts = <GIC_SPI 71 IRQ_TYPE_LEVEL_HIGH>;
313          interrupt-names = "mc";
314          phys = <&usbphy 0>;
315          phy-names = "usb";
316          extcon = <&usbphy 0>;
317          status = "disabled";
318      };
319
320  ✓      usbphy: phy@1c19400 {
321          compatible = "allwinner,sun8i-v3s-usb-phy";
322  ✓      reg = <0x01c19400 0x2c>,
323          <0x01c1a800 0x4>;
324  ✓      reg-names = "phy_ctrl",
325          "pmu0";
326          clocks = <&ccu CLK_USB_PHY0>;
327          clock-names = "usb0_phy";
328          resets = <&ccu RST_USB_PHY0>;
329          reset-names = "usb0_reset";
330          status = "disabled";
331          #phy-cells = <1>;
332      };

```

通过查询对应的设备树文档可以得知，usbphy 节点只要打开即可，usb_otg 节点还需要选择工作模式。这里我们作为主控机选择 host 模式。

```

&usb_otg {
    dr_mode = "host";
    status = "okay";
};

&usbphy {
    status = "okay";
};

```

同时，还需要在 USB 总线上兼容 OCHI (USB1.1) 和 EHCI (USB2.0) 协议。在 SOC 节点中添加下面的内容：

```

ehci0: usb@01c1a000 {
    compatible = "allwinner,sun8i-v3s-ehci", "generic-ehci";

```

```

reg = <0x01c1a000 0x100>;
interrupts = <GIC_SPI 72 IRQ_TYPE_LEVEL_HIGH>;
clocks = <&ccu CLK_BUS_EHCI0>, <&ccu CLK_BUS_OHCI0>;
resets = <&ccu RST_BUS_EHCI0>, <&ccu RST_BUS_OHCI0>;
status = "okay";
};

ohci0: usb@01c1a400 {
    compatible = "allwinner,sun8i-v3s-ohci", "generic-ohci";
    reg = <0x01c1a400 0x100>;
    interrupts = <GIC_SPI 73 IRQ_TYPE_LEVEL_HIGH>;
    clocks = <&ccu CLK_BUS_EHCI0>, <&ccu CLK_BUS_OHCI0>,
    <&ccu CLK_USB_OHCI0>;
    resets = <&ccu RST_BUS_EHCI0>, <&ccu RST_BUS_OHCI0>;
    status = "okay";
};

```

重新编译设备树并刷入，即可启用 USB 总线。

8.6 音频

8.6.1 原理图



从原理图中可以看到开发板总共有一个 mic 和一个耳机接口，而这些都是通过 v3s 的多媒体计算单元控制的。

8.6.2 驱动移植

查询 v3s.dtsi 头文件，可以看到有关音频的设备树已经有了：

```

467     codec: codec@1c22c00 {
468         #sound-dai-cells = <0>;
469         compatible = "allwinner,sun8i-v3s-codec";
470         reg = <0x01c22c00 0x400>;
471         interrupts = <GIC_SPI 29 IRQ_TYPE_LEVEL_HIGH>;
472         clocks = <&ccu CLK_BUS_CODEC>, <&ccu CLK_AC_DIG>;
473         clock-names = "apb", "codec";
474         resets = <&ccu RST_BUS_CODEC>;
475         dmas = <&dma 15>, <&dma 15>;
476         dma-names = "rx", "tx";
477         allwinner,codec-analog-controls = <&codec_analog>;
478         status = "disabled";
479     };
480
481     codec_analog: codec-analog@1c23000 {
482         compatible = "allwinner,sun8i-v3s-codec-analog";
483         reg = <0x01c23000 0x4>;
484     };

```

查询相关设备树文档，得知除了启用设备，还需要写一下音频设备路由，最终设备树呈现如下：

```

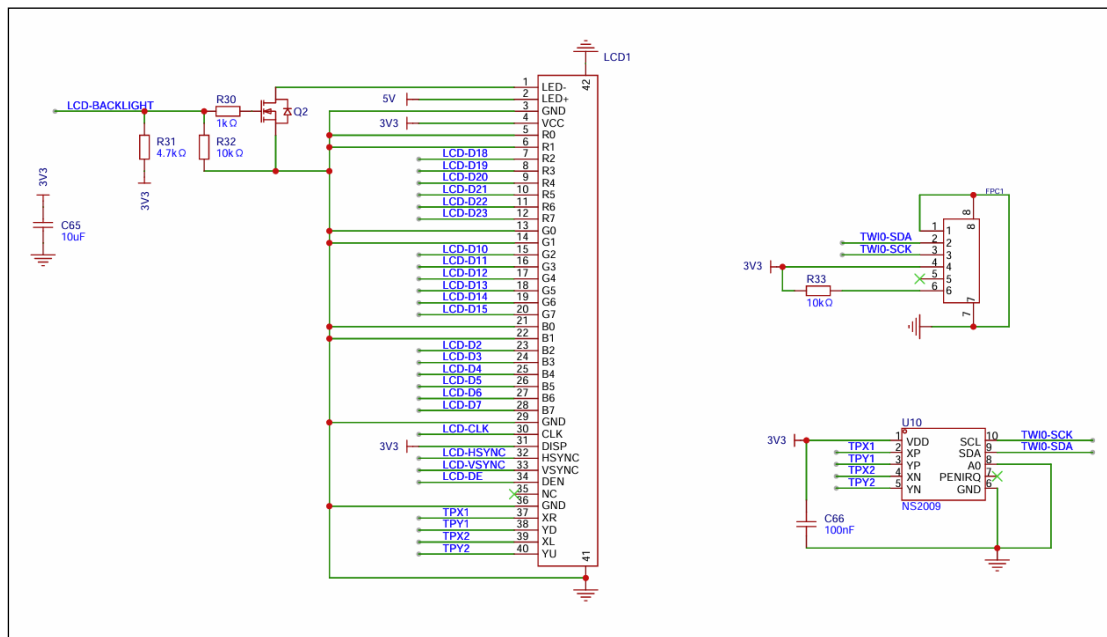
&codec {
    status = "okay";
    allwinner,audio-routing =
        "Headphone", "HP",
        "Headphone", "HPCOM",
        "MIC1", "Mic",
        "Mic", "HBIAS";
};

```

重新编译设备树并刷入，即可启用音频设备。

8.7 触控 LCD

8.7.1 原理图



从原理图中可以看到，触摸屏分两个部分，一部分是 LCD 显示，一部分是 IIC 触摸。触摸又分电容和电阻，这里我们采用开发板自带的 ns2009 电阻屏触摸芯片为例子讲解驱动移植。

8.7.2 驱动移植

8.7.2.1 LCD 显示面板

LCD 显示面板我们采用的型号为 cj050qgh50_05442y，是一块 800x480 的 LCD。驱动我们就采用 Linux 主线中自带的 panel-simple 驱动。这款驱动可以通用的驱动标准接口的 LCD 面板，只是没有我们的型号，所以我们首先要修改驱动添加自己的面板型号。如下图所示，我们添加这块面板的参数声明和兼容性声明：


```

4036 static const struct drm_display_mode cj050qgh50_05442y_mode = {
4037     .clock = 60000,
4038     .hdisplay = 800,
4039     .hsync_start = 800 + 209,
4040     .hsync_end = 800 + 209 + 30,
4041     .htotal = 800 + 209 + 30 + 16,
4042     .vdisplay = 480,
4043     .vsync_start = 480 + 22,
4044     .vsync_end = 480 + 22 + 1,
4045     .vtotal = 480 + 22 + 1 + 22,
4046     .flags = DRM_MODE_FLAG_NHSYNC | DRM_MODE_FLAG_NVSYNC,
4047 };
4048
4049 static const struct panel_desc |cj050qgh50_05442y = {
4050     .modes = &cj050qgh50_05442y_mode,
4051     .num_modes = 1,
4052     .bpc = 8,
4053     .size = {
4054         .width = 108,
4055         .height = 65,
4056     },
4057     .bus_format = MEDIA_BUS_FMT_ARGB8888_1X32,
4058     .bus_flags = DRM_BUS_FLAG_DE_HIGH | DRM_BUS_FLAG_PIXDATA_SAMPLE_POSEDGE,
4059 };

```

```

4540 }, {
4541     .compatible = "sh,cj050qgh50_05442y",
4542     .data = &cj050qgh50_05442y,

```

然后在设备树中添加新的 Panel 节点:

```

backlight: backlight {
    compatible = "pwm-backlight";
    pwms = <&pwm 0 1000000 0>;
    brightness-levels = <0 30 40 50 60 70 100>;
    default-brightness-level = <6>;
};

panel: panel {
    compatible = "sh,cj050qgh50_05442y", "simple-panel";
    backlight = <&backlight>;

    port {
        panel_input: endpoint {
            remote-endpoint = <&tcon0_out_lcd>;
        };
    };

    panel-timing {
        clock-frequency = <60000000>;
        hactive = <800>;
    };
};

```

```

        vactive = <480>;

        hfront-porch = <209>;
        hsync-len = <30>;
        hback-porch = <16>;

        vback-porch = <22>;
        vsync-len = <1>;
        vfront-porch = <22>;

        hsync-active = <0>;
        vsync-active = <0>;
        de-active = <1>;
        pixelclk-active = <0>;

    };
};

```

这样我们就完成了显示面板的设备树编写。

8.7.2.2 LCD 控制器

注意到上面的面板设备树依赖了一个未知节点 `tcon0_out_lcd`, 这个就是 v3s 内置的 LCD 显示控制器。实际上只启用面板并不能正常显示图像, 因为 Linux 不知道如何控制这个面板显示, 这里我们还需要给面板上游配置一个 LCD 控制器。好在 v3s 内置了一个这样的控制器, 我们不需要搭配第三方芯片。

从 Linux 的显示信号出来到 LCD 面板, 中间不止一个节点。具体到我们的开发板, 总共需要下面的几个节点:

Display Engine → Mixer → T_CON → LCD Panel

所以说除了我们刚刚写的 LCD Panel, 我们还需要编写上面的几个节点的设备树。不过在哪些节点在 `sun8i-v3s.dtsi` 文件中都编写完了, 我们只要挨个启用即可

```

&de {
    status = "okay";
};

&pwm {
    pinctrl-names = "default";
    pinctrl-0 = <&pwm0_pins>;
    status = "okay";
};

&pio {

```

```

    lcd_pins: lcd-pins {
        pins = "PE0", "PE1", "PE2", "PE3", "PE4", "PE5",
               "PE6", "PE7", "PE8", "PE9", "PE10", "PE11",
               "PE12", "PE13", "PE14", "PE15", "PE16", "PE17",
               "PE18", "PE19", "PE23", "PE24";
        function = "lcd";
    };

    pwm0_pins: pwm0-pins {
        pins = "PB4";
        function = "pwm0";
    };
};

&tcon0 {
    pinctrl-names = "default";
    pinctrl-0 = <&lcd_pins>;
    status = "okay";
};

&tcon0_out {
    tcon0_out_lcd: endpoint@0 {
        reg = <0>;
        remote-endpoint = <&panel_input>;
    };
};
};

```

即可正确启用控制器。

8.7.2.3 U-boot 源码修改

除了硬件设施之外,我们还需要提前预留一块内存作为显存,这段显存称为FrameBuffer。这里有两种实现方式,1 是从 u-boot 引导阶段预留,并直接传递给内核,2 是直接在内核设备树声明。这里我们采用方式 1。

在 dtsti 头文件中,chosen 节点已经声明了预留的 buffer, 如下图:

```

55     chosen {
56         #address-cells = <1>;
57         #size-cells = <1>;
58         ranges;
59
60         framebuffer-lcd {
61             compatible = "allwinner,simple-framebuffer",
62             | "simple-framebuffer";
63             allwinner,pipeline = "mixer0-lcd0";
64             clocks = <&display_clocks CLK_MIXER0>,
65             | <&ccu CLK_TCON0>;
66             status = "disabled";
67         };
68     };

```

但是没有地址信息。这个 buffer 预留的地址是需要从 uboot 传递的。所以我们这里同样要修改 uboot 的源码。

要从 u-boot 添加 LCD 支持，需要从一个很久以前的 u-boot 分支中拉取 v3s 的面板支持并合并（https://github.com/mcerveny/u-boot/tree/simplefb_v3s_v2）。之后再添加设备树如下：

```

/ {
    backlight: backlight {
        compatible = "pwm-backlight";
        pwms = <&pwm 0 1000000 0>;
        brightness-levels = <0 30 40 50 60 70 100>;
        default-brightness-level = <6>;
    };

    panel: panel {
        compatible = "sh,cj050qgh50_05442y", "simple-panel";
        backlight = <&backlight>;

        port {
            panel_input: endpoint {
                remote-endpoint = <&tcon0_out_lcd>;
            };
        };

        display-timings {
            timing0: timing0 {
                clock-frequency = <60000000>;
                hactive = <800>;
                vactive = <480>;
            };
        };
    };
}

```

```
        hfront-porch = <209>;
        hsync-len = <30>;
        hback-porch = <16>;

        vback-porch = <22>;
        vsync-len = <1>;
        vfront-porch = <22>;

        hsync-active = <0>;
        vsync-active = <0>;
        de-active = <1>;
        pixelclk-active = <0>;

    };
};
};
};
&pwm {
    pinctrl-names = "default";
    pinctrl-0 = <&pwm0_pins>;
    status = "okay";
};

&pio {
    lcd_pins: lcd-pins {
        pins = "PE0", "PE1", "PE2", "PE3", "PE4", "PE5",
               "PE6", "PE7", "PE8", "PE9", "PE10", "PE11",
               "PE12", "PE13", "PE14", "PE15", "PE16", "PE17",
               "PE18", "PE19", "PE23", "PE24";
        function = "lcd";
    };

    pwm0_pins: pwm0-pins {
        pins = "PB4";
        function = "pwm0";
    };
};

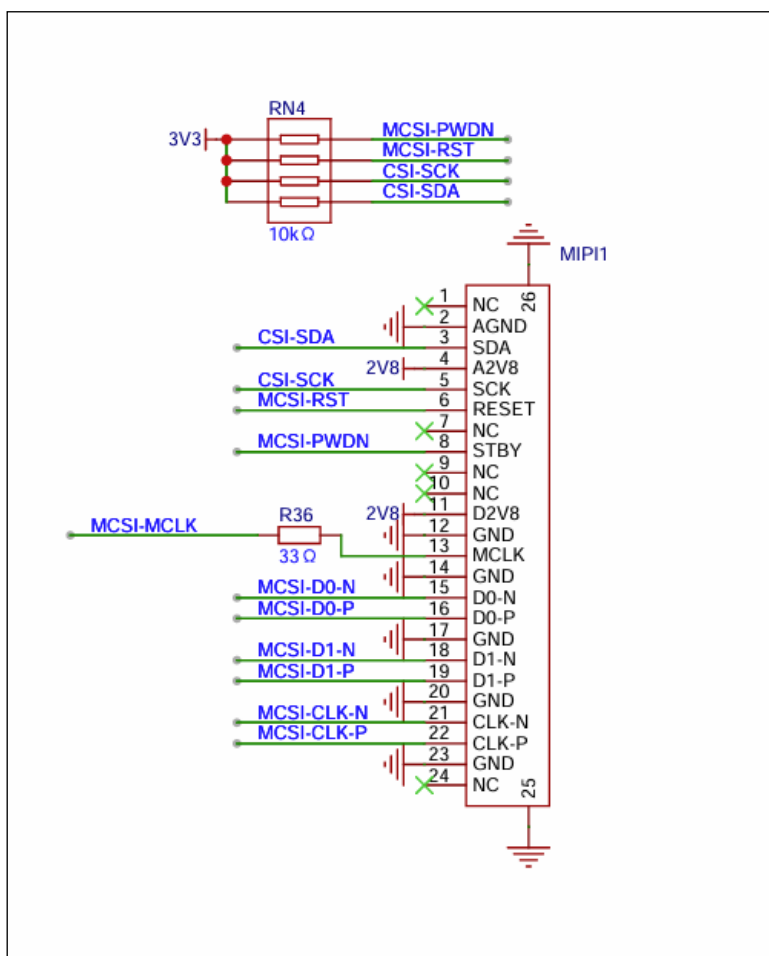
&tcon0 {
    pinctrl-names = "default";
    pinctrl-0 = <&lcd_pins>;
    status = "okay";
};
```

```
&tcon0_out {
    tcon0_out_lcd: endpoint@0 {
        reg = <0>;
        remote-endpoint = <&panel_input>;
    };
};
```

即可实现 u-boot 的 LCD 支持,并且 u-boot 在加载完成后,会将 framebuffer 传递给 Linux 内核。

8.8 摄像头

8.8.1 原理图



V3s 芯片共有两种摄像头输入，一个是与 LCD 复用的 DVP 输入，这个我们已经无法使用了；另一个是使用 MIDI-CS12 的摄像头输入接口，我们的开发板使用了这一个接口。摄像头接口预留需要外接摄像头，这里我们采用 ov5647 型号传感器的摄像头为例。

8.8.2 驱动移植

摄像头驱动也分为多个节点，并且摄像头的相关资料全志手册中并没有公开，这里采用网络上逆向得到的寄存器资料。摄像头数据传输流程为：

Ov5647 传感器→mipi_dphy 物理接口→mipi_csi2 接口→csi0 总线

所以我们共需要写四个设备树节点。这些节点驱动 Linux 主线中全都有，所以我们只需要添加设备树节点即可：

```
/ {
    soc {
        csi0: camera@1cb0000 {
            compatible = "allwinner,sun8i-v3s-csi";
            reg = <0x01cb0000 0x1000>;
            interrupts = <GIC_SPI 83 IRQ_TYPE_LEVEL_HIGH>;
            clocks = <&ccu CLK_BUS_CSI>,
                <&ccu CLK_CSI1_SCLK>,
                <&ccu CLK_DRAM_CSI>;
            clock-names = "bus", "mod", "ram";
            resets = <&ccu RST_BUS_CSI>;
            interconnects = <&mbus 5>;
            interconnect-names = "dma-mem";
            status = "okay";

            ports {
                #address-cells = <1>;
                #size-cells = <0>;

                port@1 {
                    reg = <1>;

                    csi0_in_mipi_csi2: endpoint {
                        remote-endpoint = <&mipi_csi2_out_csi0>;
                    };
                };
            };
        };
        mipi_csi2: csi@1cb1000 {
            compatible = "allwinner,sun8i-v3s-mipi-csi2",
                "allwinner,sun6i-a31-mipi-csi2";
            reg = <0x01cb1000 0x1000>;
            interrupts = <GIC_SPI 90 IRQ_TYPE_LEVEL_HIGH>;
            clocks = <&ccu CLK_BUS_CSI>,
                <&ccu CLK_CSI1_SCLK>;
            clock-names = "bus", "mod";
            resets = <&ccu RST_BUS_CSI>;
```

```
        status = "okay";

        phys = <&dphy>;
        phy-names = "dphy";

        ports {
            #address-cells = <1>;
            #size-cells = <0>;

            mipi_csi2_in: port@0 {
                reg = <0>;
                mipi_csi2_in_sensor: endpoint {
                    data-lanes = <1 2>;
                    clock-lanes = <0>;
                    remote-endpoint = <&ov5647_out_mipi_csi2>;
                };
            };

            mipi_csi2_out: port@1 {
                reg = <1>;

                mipi_csi2_out_csi0: endpoint {
                    remote-endpoint = <&csi0_in_mipi_csi2>;
                };
            };
        };
};

dphy: d-phy@1cb2000 {
    compatible = "allwinner,sun6i-a31-mipi-dphy";
    reg = <0x01cb2000 0x1000>;
    clocks = <&ccu CLK_BUS_CSI>,
            <&ccu CLK_MIPI_CSI>;
    clock-names = "bus", "mod";
    resets = <&ccu RST_BUS_CSI>;
    allwinner,direction = "rx";
    status = "disabled";
    #phy-cells = <0>;
};

&i2c1 {
    status = "okay";
    pinctrl-names = "default";
```



```
pinctrl-0 = <&i2c1_pe_pins>;
ov5647: camera@36 {
    compatible = "ovti,ov5647";
    reg = <0x36>;
    clocks = <&ccu CLK_CSI0_MCLK>;

    port {
        ov5647_out_mipi_csi2: endpoint {
            remote-endpoint = <&mipi_csi2_in_sensor>;
        };
    };
};
};
```

然后在内核中选中相关驱动，即可使用摄像头。